Session 3.1

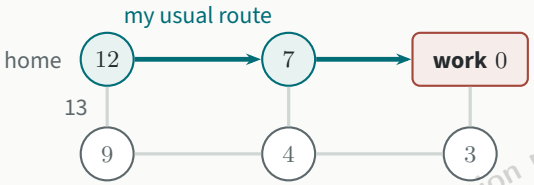
From Dynamic Programming to Reinforcement Learning



Zhigang Feng

© Zhigang Feng
Redistribution prohibited without permission

The Setup: a Map of Turns, and No Travel Times



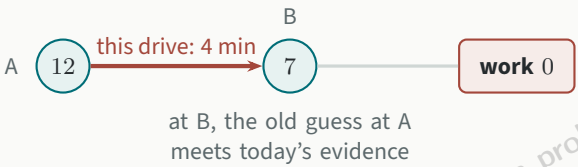
a guess per turn: from home 12 straight on, 13 down; circles show the usual turn's

known: which turns exist
unknown: minutes per street; traffic, which varies by the day
goal: a habit, one turn a corner, fewest minutes to work

© Zhigang Feng. Redistribution prohibited without permission

- New city: the map shows which turns exist; nobody knows the minutes; traffic varies.
- The goal is a habit: a turn at every corner that reaches work in the fewest minutes.
- A guess per turn at every corner: minutes to go if I take it, then drive as usual. The circle shows my usual turn's.
- A drive reveals, corner by corner: where I was, the turn, the minutes, where I landed.
- **Everything the learner will ever use is in those four things. The map only lists the options.**

How We Play: Drive, Compare, Correct a Little

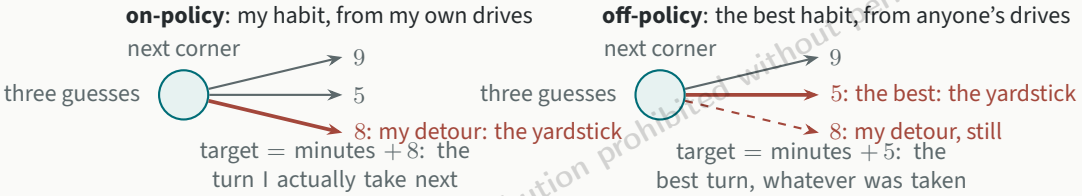


target $= 4 + 7 = 11$
surprise $= 11 - 12 = -1$
new guess $= 12 + \text{step} \times (-1)$
step $\frac{1}{2}$: $11\frac{1}{2}$; step 1: 11; step $1/n$: the average of all n drives

© Zhiang Fan. Redistribution prohibited without permission

- Drive. At each corner take the turn with the smaller guess; now and then, any turn.
- The target at the next corner: minutes just spent plus the guess there, $4 + 7 = 11$.
- Move the old guess part of the way: the step is how much of one drive you believe; today's 4 minutes may be luck, and B's 7 is itself a guess.
- Step $\frac{1}{2}$: $11\frac{1}{2}$. Step 1 takes today whole: 11. Step $1/n$ on the n-th visit: the average of every drive, Deck 3.3's running mean.
- **Compare, correct a little, keep driving. Work anchors it: zero minutes there, a fact, passed back a corner a day.**

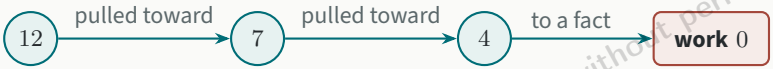
Two Ways to Learn: On-Policy and Off-Policy



© Logan Fehr. Redistribution prohibited without permission

- Both drive alike; all three numbers are guesses; only the yardstick differs.
- On-policy: the yardstick is the turn I take next, detour or not. I learn my own habit's value: precise, but a habit that detours is slightly wrong everywhere.
- Off-policy: the yardstick is the best turn there, whatever I take: the best habit's value; but the best of noisy guesses is optimistic, and it is checked only when driven.
- **Same drives, two questions: my habit's value, answered precisely; the best habit's, answered noisily.**

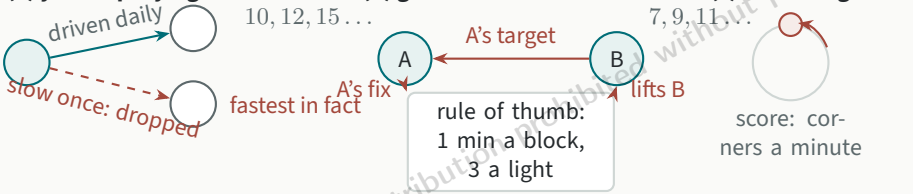
Why the Guesses Settle: Every Yardstick Is Itself Corrected



© Shuguang Feng · Redistribution prohibited without permission

- Each corner is pulled toward the corners it leads to; the last, to the real arrival.
- You correct the corners you drive through, as often as you drive them (on-policy): every yardstick is a corner you also correct.
- So the truth travels back from work one corner at a time: the web tightens like a net drawn in to the shore.
- The classic success, TD-Gammon: backgammon learned by self-play; every position it judged by, it also corrected, and every game ended in a real result.
- **Guesses checked against guesses are safe when they are checked where they are used.**

Why It Can Fail: Three Ways



© Eugene Feinberg, Unauthorized distribution prohibited without permission

- Stop trying: a street slow once is never driven again, and the data stay silent on it. Cure: a worse-looking turn now and then; free in a simulator, costly in an economy.
- Guesses chase each other: a rule of thumb makes one fix move every guess. Correct A against B's guess, never B itself: each fix lifts B, which lifts A's target: a runaway.
- Wrong score: it does what you wrote, not what you meant. Corners a minute? It circles.
- **Three dangers, three remedies to come: keep trying (Deck 3.2); close the loop or freeze the target (Decks 3.3–3.4); grade against a known answer (Deck 3.5).**

The Same Story in This Course's Words

the story	the word	the growth model
a corner	state	capital and productivity, (k, z)
a turn	action	how much to save, k'
minutes spent	reward	this period's utility, $u(c)$
minutes to go	value	lifetime utility from here, $V(k, z)$
the city	the environment	the model, stepped one period
the map of turns	what is known	the law of motion of capital
the times and the traffic	what is learned	the shocks; later, the distribution's motion

© Lisang Feng · Redistribution, modification without permission

- The driver knows the map and learns the rest. Here the law of motion is known; the shocks are learned.
- **Session 3 in one line: two habits (Decks 3.2–3.3), a rule of thumb (Deck 3.4), every method on one model (Deck 3.5). Three verdicts next: on or off; what we use RL for; why not in the economy.**

On-Policy or Off-Policy: Neither Is Better

	on-policy	off-policy
whose value	the rule you run, detours included	the best rule, or any rule you name
data	its own fresh drives only	anyone's: old records, a replay memory
the best rule	reached as the detours fade	reached while still detouring
noise	an average's	a max or reweighting: optimistic
a rule of thumb	stable: every yardstick corrected	can chase itself (p. 6)
needs	freedom to run the rule and tremble	overlap: data that do what it would

© Jiarang Feng · Redistribution prohibited without permission

- On wins when you can run the rule and tremble freely; a rule of thumb stays stable.
- Off-policy wins when data are scarce or someone else's, or the best rule is wanted now.
- **An accurate value of a slightly wrong rule, or a noisy value of the right rule from anyone's data (Deck 3.2). In an economy the choice is made for you: off.**

How Economists Use Reinforcement Learning: Four Ways

a solver in a written model	**a model of behaviour**	**policy evaluation** from data	**adaptive experiments**
heterogeneous agents, a distribution in the state	agents who learn; pricing algorithms that collude	overlap, inverse probability weights, policy learning	bandits on platforms: tests that adapt

© Zhirang Feng · Redistribution prohibited without permission

- The mathematics was ours first: Bellman, Howard, Monte Carlo, adaptive expectations.
- As a solver the model is the simulator, trembles free, drives unlimited: RL scales a model too big for a grid (Maliar–Maliar–Winant; Azinovic–Gaegauf–Scheidegger).
- As behaviour: learning, experimenting agents; algorithms that collude (Calvano et al.).
- As evaluation: RL's coverage is overlap; same weights, same fixes (Athey–Wager).
- **RL's mathematics is ours; its algorithms are becoming our solvers; its one new idea is used where the environment is a model (Deck 3.5).**

Why Not in the Real Economy: It Is Not a Simulator

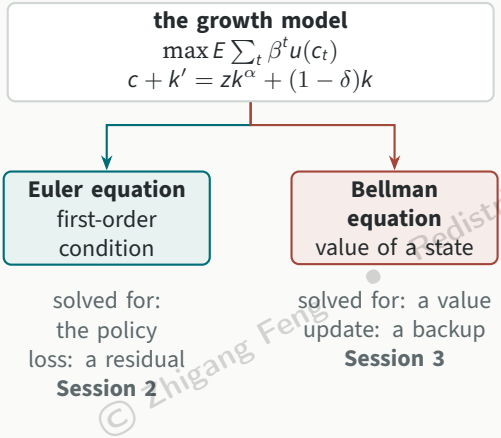
a simulator
a reset button, free trembles, unlimited drives; responses fixed: physics, dice, traffic

the economy
one history, once; a tremble is a policy error; decades for one rule's data; responses move with the rule

© Zhang Fan. Redistribution prohibited without permission

- A central bank runs one rule, once, for real: it cannot tremble.
- So any new rule is valued off-policy, with thin overlap where the old rule never did what the new one would; and Lucas: even with overlap, the responses were the old regime's.
- Identification needs a model: the data saw one rule; only an assumption separates what stays from what moves. Estimate it, re-solve, simulate (Clarida–Galí–Gertler).
- Engineering has the data limits, not Lucas: a turbine expects nothing of its controller, so old data and simulators stay faithful; RL's wins are games and simulated robots.
- **RL scales the solution of a model; it does not replace the model. Now the road in from Session 2, and the three challenges.**

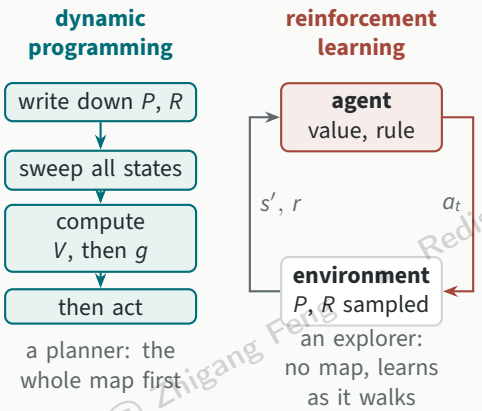
Session 2 Took the Euler Road. The Bellman Equation Is the Other One



© Zhigang Feng — Redistribution prohibited without permission

- Session 2 trained a policy network on Euler residuals, never a value.
- The Bellman equation describes the same optimum through the *value* of each state.
- It needs no derivatives: discrete choices and binding constraints are fine.
- **This deck takes that road and meets three challenges: the state grows, the backup looks further ahead, the law of motion has no closed form.**

Two Ways to Solve the Same Bellman Equation: Model First, or Learn by Acting

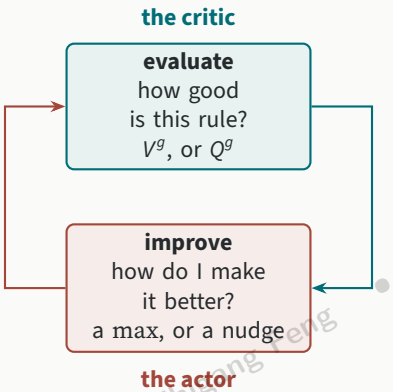


- Dynamic programming writes down P and R, sweeps every state, computes V, then acts.
- **Reinforcement learning** acts, sees a reward and a new state, improves, acts again.
- **RL's one idea**: learn from realised (s, a, r, s'), one at a time, made by its own rule. A learner that reads only these, never u or P_z, is **model-free**.
- **RL fills the expectation with samples: each challenge ahead leaves only states and transitions along walked paths.**

© Zhigang Feng Redistribution prohibited without permission

Source: R. Sutton, A. Barto, *Reinforcement Learning: An Introduction*, 2nd ed., 2018, ch. 1.

Two Questions, Two Learners



- Evaluate to convergence, then improve: Howard's (1960) **policy iteration**. One backup, then improve: value function iteration.
- The **critic** evaluates; the **actor** improves. **A structure, not a source of data.**
- Methods differ in what each learner reads (the model, or experience) and how far ahead it looks (one period, T, the whole path).
- **Every later deck answers both in turn, on different data: Deck 3.2 from whole paths, 3.3 from single steps, 3.4 with the model in hand, then with a network per learner. They return once more with the Third Answer (p. 25).**

© Zhengyang Feng
Redistribution prohibited without permission

Session 3.1

The Benchmark, and What It Requires

The growth model's Bellman equation, and the two things dynamic programming needs to solve it



© Zhigang Feng · Redistribution prohibited without permission

The Growth Model: a Sequence Problem, and Its Recursive Form

sequence problem

$$\max E_0 \sum_t \beta^t \ln c_t$$

$$c_t + k_{t+1} = y(k_t, z_t)$$

one function, not a path

Bellman equation

$$V = \max_{k'} \{\ln c + \beta E[V' \mid z]\}$$

$y(k, z) = z\, f(k) + (1 - \delta)k$, $f(k) = k^\alpha$: output plus capital left

- Sequential, coupled, uncertain; yet today's utility plus the same problem from tomorrow, since z is Markov: one equation.
- k real, so is k'; or a **saving share** $a \in (0, 1)$: $k' = a\, y$, $c = (1 - a)y$. Two shocks, 0.9 and 1.1, staying w.p. 0.8.
- T^*, the **Bellman operator** (the star: a $\max$ inside), contracts with modulus β (Session 1.4): V^* unique, iteration converges, g^* its $\arg\max$.

© Zhuang Feng • Redistribution prohibited without permission

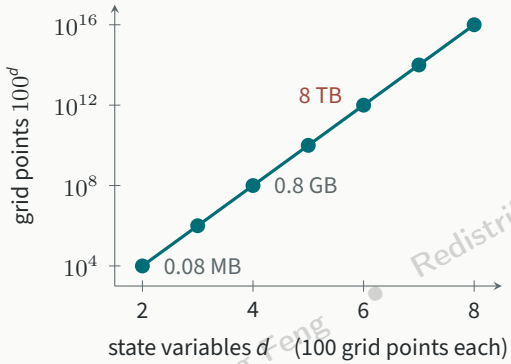
What Dynamic Programming Quietly Requires

Requirement	In value iteration, each round	Where it fails
a table, swept	V kept at every grid point of (k, z), interpolated off it; every point updated, every round	m^d entries with d state variables: too many to store, too many to sweep
the model: score, average	P_z and the law of motion inside $E[V' \mid z]$: each k' scored, tomorrow averaged	a sum too large to take; a law of motion that is not known

- Two requirements: a store that can be swept, and a model to score and average.
- In two dimensions neither binds: DP runs, and V^*, g^* are the **answer key**; the **grade** of a learned rule is the consumption given up for ever to hold it instead.
- The store breaks when the state grows (Challenge 1); the average, when the backup looks further ahead (Challenge 2); the model, when the law of motion has no closed form (Challenge 3).
- **Three challenges, three answers: a network for the table, a walk for the sum, a simulated economy for the model. One at a time, next.**

© Albert Pell. Undistributed without permission prohibited

Challenge 1: the Curse of Dimensionality in the State, Which RL Still Faces

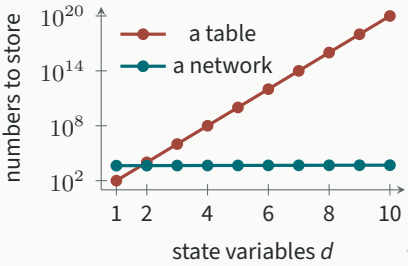


© Zhigang Feng
Redistribution prohibited without permission

- m points per dimension: m^d entries; one more variable multiplies them by m.
- Go: rules on one page, about 10^{170} positions. A known model, and no table.
- *Sampling changes which states get visited, not the table's size.*
- **A table shares nothing between its entries. The first answer: a function, where one parameter serves many states.**

Source: Memory at 8 bytes per entry; the count of legal Go positions: J. Tromp, 2016.

First Answer: a Network for the Table

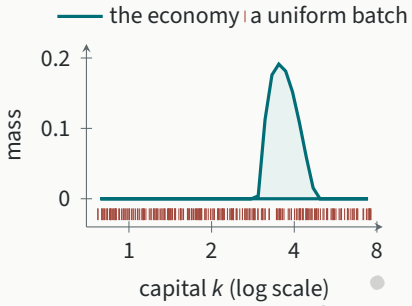


- A table needs a number for every state; a network grows with d only in its first layer.
- Fitted by regression on a sample of states, it needs to be right only where the sample falls: next page.
- **Projection** (a fit on a basis) is economists' version; **deep DP** (by networks) the same tool at larger d. A poor form spreads its errors (Deck 3.4).
- **First answer: a network for the table. Its size grows with d, not m^d, and its fit is paid only where it is needed.**

© Zhigang Feng
Redistribution prohibited without permission

Source: Table: 100 points a variable, 8 bytes a number. Network: d inputs, two hidden layers of 64 units, one output.

The First Answer, Completed: Fit It Where the Economy Goes

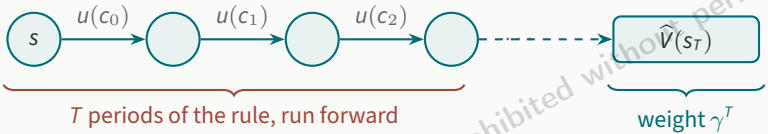


- A uniform box is cursed too: six times wider in $\ln k$ than where the economy spends 98 % of its time; 6^d times the volume in d dimensions.
- **Simulate, burn in, draw**: the rule's path picks the states, as **ergodic-set methods** do.
- Fitted on that region, not the box: the cost of approximation is the region's size.
- Its price: rare states fit worse, and the sample moves with the rule (Deck 3.4).
- **Challenge 1 answered twice: a network, fitted where the economy goes, not the box.**

© Zhigang Feng
Redistribution prohibited without permission

Source: Session 3 lab, block B: the long-run distribution of k under the learned rule, $z = 0.9$; ticks: one uniform batch of 256 states.

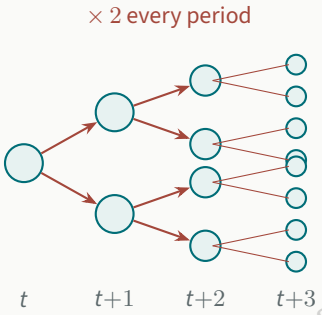
Challenge 2: Why Look Further Ahead



© Lilian Pintus. Unauthorized distribution prohibited without permission

- Value iteration's **one-period backup**, $B(s) = u(c) + \gamma \sum_{z'} P_z(z' \mid z)\, \widehat{V}(k', z')$: today's utility is real; the rest is last round's guess, at weight $\gamma = 0.95$.
- A classical speed-up: between two maximisations, run the current rule T periods forward, $B_T(s) = E\big[\sum_{t<T} \gamma^t u(c_t) + \gamma^T \widehat{V}(s_T) \,\big|\, s_0 = s\big]$: the guess's weight falls to γ^T, 0.77 at $T = 5$, 0.36 at $T = 20$. This is **modified policy iteration**, Howard's rollout.
- Only the evaluation's target lengthens; the maximisation is untouched. Fewer rounds, each trusting fewer numbers.
- **But the T-period expectation runs over T periods of shocks. From one state, the uncertainty unfolds: a tree.**

The Tree: Uncertainty Unfolds, Two Branches a Period

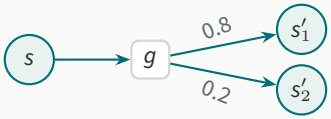


- One period ahead the backup has n_z branches, weighted by P_z: two here.
- T periods ahead, n_z^T shock histories: 2^T here, and $2^{20} \approx 10^6$ at $T = 20$. With d_z shocks of n_z values each the base is $n_z^{d_z}$: every shock added multiplies every level.
- Reading the tree exactly costs $2^{T+1} - 2$ evaluations of the rule and the model per state: 62 at $T = 5$, two million at $T = 20$.
- **A second curse, in time: the lookahead's cost grows as n_z^T, as the table's grew as m^d. The second answer: sample it.**

© Zhigang Feng
Redistribution prohibited without permission

Second Answer: Sample the Tree by Walking It

enumerate: every branch, weighted



run: one branch, drawn



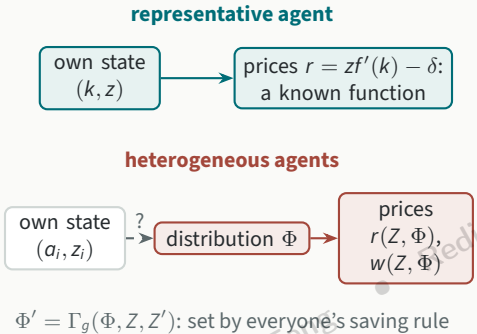
	enumerate	run
reads	P_z	realised (r, s')
T ahead	$2^{T+1} - 2$	$N \times T$
error	none	$\sigma_G/\sqrt{N}$

σ_G: one run's spread

- A **sample backup**: run the rule N times for T periods, drawing z', and average: cost $N \cdot T$, error $\sigma_G/\sqrt{N}$, free of n_z^T and of d.
- No P_z is read: the simulator is a black box returning (r, s'); with the model known, it is the model stepped once.
- **The first time the learner acts**: **the walk**, forced by the lookahead (Decks 3.2–3.3 run it).
- **RL's one idea (p. 12): learn from transitions the rule itself makes; with the model known, another path, not a last resort. The price: variance.**

© Zhigang Feng
Redistribution prohibited without permission

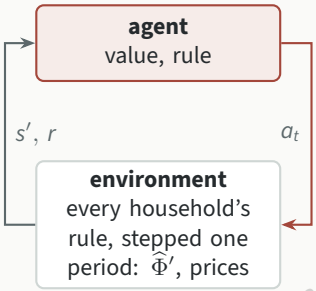
Challenge 3: an Unknown Law of Motion for the Distribution



- Heterogeneous agents, aggregate shocks: prices depend on the distribution Φ; to forecast r' is to forecast Φ', so Φ enters the state.
- Γ_g, its law of motion, has no closed form in most models and depends on the rule solved for; yet a period can always be run (Deck 3.5).
- **The most severe: not a cost, a missing input. Every primitive is written, yet part of the environment, how Φ and prices move, is unknown; in Session 4 usually truly unknown to the household.**

© Zhigang Feng. Redistribution prohibited without permission

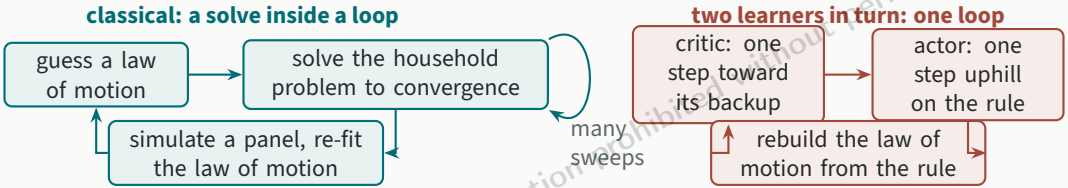
Third Answer: Learn the Environment by Running It



- No closed form, yet it runs: step every rule once, and Φ' and the prices are realised.
- A learner reading only (s, a, r, s') never needs Γ_g: the simulation stands in for it.
- It learns the law of motion where the rule goes, and re-learns it as the rule changes.
- The price: the data move with the rule; each step changes tomorrow's sample (Deck 3.4).
- **Third answer: learn the environment by simulating it; the walk is the only source.**

© Zhigang Feng • Redistribution prohibited without permission

Two Learners in Turn, So the Fixed Point Becomes Training



- Classical Krusell–Smith nests two fixed points: the household problem, solved to convergence, inside a loop that re-fits the law of motion from a simulated panel.
- Two learners in turn take one step each, the actor along a derivative (Deck 3.4); the law of motion is rebuilt by stepping the current rule: no inner solve, no outer loop.
- The three answers compose: a network that takes Φ as input (p. 18), a walk instead of a sum (p. 22), and alternation that turns the fixed point into training.
- **The third answer, made efficient: a step per learner and one simulated period, never a solve.**

© Jesse Perla. Unauthorized distribution prohibited without permission

Session 3.1

One Problem, Two Languages

The Markov decision process, its dictionary, and the two value functions every later deck uses



© Zhigang Feng · Redistribution prohibited without permission

The Markov Decision Process

- A **Markov decision process** (MDP) is a five-tuple
$$\mathcal{M} = \langle \mathcal{S},\ \mathcal{A},\ P,\ R,\ \gamma \rangle.$$
- **States** $\mathcal{S}$; feasible **actions** $\mathcal{A}(s)$; **transition** probabilities $P(s' \mid s, a)$; expected **reward** $R(s, a)$; **discount factor** $\gamma \in [0, 1)$, the economist's β from here on.
- The **Markov property**, $P(s_{t+1} \mid s_t, a_t, s_{t-1}, a_{t-1}, \ldots) = P(s_{t+1} \mid s_t, a_t)$: the current state carries everything needed to forecast.
- P: a law of motion plus shocks, or chance of its own (a job offer). DP assumes P known; RL lets it be unknown.
- **Markov makes a problem recursive, not computable: enlarge the state (habit: add c_{t-1}) and the three challenges remain.**

Source: M. Puterman, *Markov Decision Processes*, 1994.

© Zhaonan Feng · Redistribution prohibited without permission

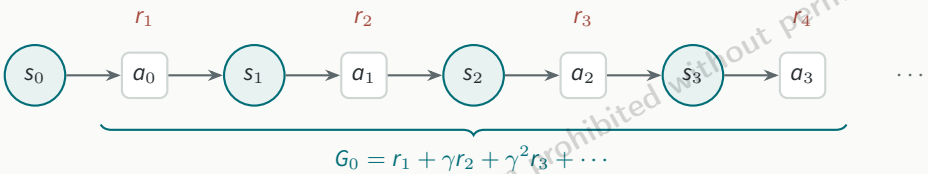
One Dictionary: the MDP and the Dynamic Economic Model

MDP	Economics	Example
state s	state variables	capital k, wealth, productivity z
action a	decision, choice variable	next capital k', the policy rate
transition $P(s' \mid s, a)$	law of motion, shocks	k' chosen, or $a\,y$; $z' \sim P_z(\cdot \mid z)$
reward $R(s, a)$	period utility or profit	$u(c)$
discount γ	discount factor β	0.99 quarterly, 0.95 annual
policy $g(a \mid s)$	decision rule	$k' = g(k, z)$
value $V^g(s)$	value function	lifetime utility $V(k, z)$
episode	a finite life, or a simulated path	a T-period run

- The policy is g (RL's π); *control* here means improving the rule (Deck 3.2), not the action.
- **Rule, policy, decision rule** are one object: the actor is the rule a learner improves, the critic the value it keeps; Session 2's share s is here the action a.
- **The heterogeneous-agent household is this five-tuple with state $(a_i, z_i; Z, \Phi)$ and Γ_g inside P: Markov in form, infinite-dimensional to compute (p. 23).**

© Zhiyang Feng · Redistribution prohibited without permission

Trajectory and Return



- A **trajectory** is one simulated path. The reward r_{t+1} is period t's utility $u(c_t)$; the index says when it is seen.
- The **return** is the discounted utility stream, $G_t = \sum_{j \geq 0} \gamma^j r_{t+j+1}$.
- It is recursive: $G_t = r_{t+1} + \gamma\, G_{t+1}$. Take expectations and the Bellman equation appears.
- Only T periods simulated: the **partial return** $G_{t:t+T} = \sum_{j<T} \gamma^j r_{t+j+1}$ (Decks 3.2–3.3).
- **A return is one random draw of lifetime utility. A value is its expectation.**

Source: Sutton & Barto, *Reinforcement Learning*, 2018, §3.3.

© Zhigang Feng Redistribution prohibited without permission

Two Value Functions: V and Q

- The **state-value function**: expected return from state s when following g,
$$V^g(s) = E_g\big[\, G_t \mid s_t = s \,\big].$$
- The **action-value function**, or Q **function**: take action a first, follow g afterwards; on a grid a table, one row per state (Deck 3.2 prints Q^*),
$$Q^g(s,a) = E_g\big[\, G_t \mid s_t = s,\ a_t = a \,\big].$$
- Linked by $V^g(s) = \sum_a g(a \mid s)\, Q^g(s,a)$. The **Bellman expectation equation** evaluates a *given* rule, with no $\max$. Call its right-hand side $T^g V$:
$$V^g(s) = R(s, g(s)) + \gamma \sum_{s'} P(s' \mid s, g(s))\, V^g(s').$$
- T^g is a contraction of modulus γ, as T^* is (p. 15). With V alone, improving needs the expectation for every action; one sample shows one, so a sampling learner keeps Q, or an actor.
- **The Bellman equation never changes this session; only how its expectation is filled: summed, or sampled from the walk.**

© Zhigang Feng · Redistribution prohibited without permission

Session 3.2

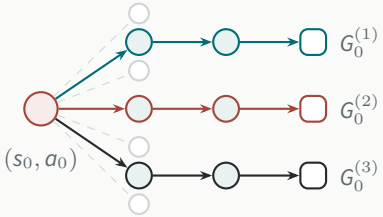
Monte Carlo: Evaluation, Control, and the Data



Zhigang Feng

© Zhigang Feng
Redistribution prohibited without permission

Basic Monte Carlo: Walk N Paths from One Cell, Average the Returns



Basic Monte Carlo for one cell

input: g, (s, a), N paths, horizon T
for $i = 1, \ldots, N$:
  simulate T periods: $a_0 = a$, then g
  $G_0^{(i)} \leftarrow \sum_{j=0}^{T-1} \gamma^j r_{j+1}$
return $\widehat{Q}^g(s, a) \leftarrow \frac{1}{N} \sum_i G_0^{(i)}$

- **Monte Carlo** (MC): without P_z, simulate N lifetimes from (s, a), Deck 3.1's walk run to the end with no $\widehat{V}$ to end on, and average their discounted utility.
- Each path is cut at $T = 300$ periods, where the discounted tail no longer matters; the test bed is the smallest instance, the 7×2 grid.
- **Cost per cell $O(NT)$, whatever the number of states or shock histories.**

© Zhigang Feng · Redistribution prohibited without permission

The Exact Q^* Table: the Answer Key

$k \setminus k'$	1.50	2.25	3.00	3.75	4.50	5.25	6.00
1.50	**0.563**	-0.004	–	–	–	–	–
2.25	1.226	**1.932**	1.441	–	–	–	–
3.00	1.609	2.533	**2.946**	2.414	–	–	–
3.75	1.879	2.898	3.511	**3.735**	3.054	–	–
4.50	2.088	3.160	3.864	**4.277**	4.275	3.620	–
5.25	2.259	3.366	4.122	4.623	**4.803**	4.784	4.126
6.00	2.403	3.534	4.325	4.877	5.144	**5.303**	5.257

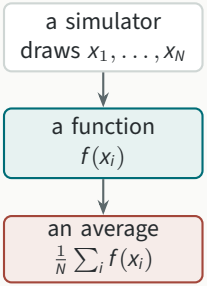
Redistribution prohibited without permission

- The $z = 0.9$ half of the 7×2 grid, from DP. Bold: the row maximum, the optimal k': Deck 3.1's answer key.
- Two rows to look at. At $k = 3.75$ the best choice wins by 0.224, 1.13% of consumption for ever; at $k = 4.50$ by 0.002, a hair. **Big gaps are easy to learn; hair-thin gaps are what sampling gets wrong, and they cost little.**

© Zhigeng Fang

Source: Session 3 lab, block A: the 7×2 grid, $\gamma = 0.95$; in consumption, $\exp\{(1-\gamma)\Delta\} - 1$.

What Monte Carlo Is: Simulate and Average



shocks, paths, households

a moment, a return

$\rightarrow E[f(X)]$

- One simulated life is luck; average many and it cancels: the mean tends to $E[f(X)]$.
- Its error falls as $1/\sqrt{N}$, whatever the number of states; close values need many paths.
- **Economists already use it**: the simulated method of moments, same estimator, another f.
- Here f is a lifetime return, and the average improves a *rule*, not a parameter.
- **The estimator is old. New is what gets averaged, and what for.**

© Zhigang Feng
Redistribution prohibited without permission

Dynamic Programming or Monte Carlo: When to Use Which

	Dynamic programming	**Monte Carlo**
needs	P, summed at every state	a simulator: only paths
updates	every state, every sweep	visited cells, after a path ends
error	none from sampling	variance, falling like $1/\sqrt{N}$
cost, d variables	m^d states, a sum at each	N paths a cell at any d; still a cell per pair

- **Sampling replaces the sum over tomorrow, not the table**: one cell costs the same at any d, but there is still a cell per state–action pair (Deck 3.4 drops the table).
- **Stay with DP** when the state can be swept and the expectation summed; **leave it when either fails**.
- From here to Deck 3.4's model-free learners we pretend not to know P_z; in Session 4 the distribution's law of motion is usually truly unknown to the household.
- **Model-free is not error-free: a misspecified simulator passes its error straight on.**

© Jiaqiang Feng · Redistribution prohibited without permission

Session 3.2

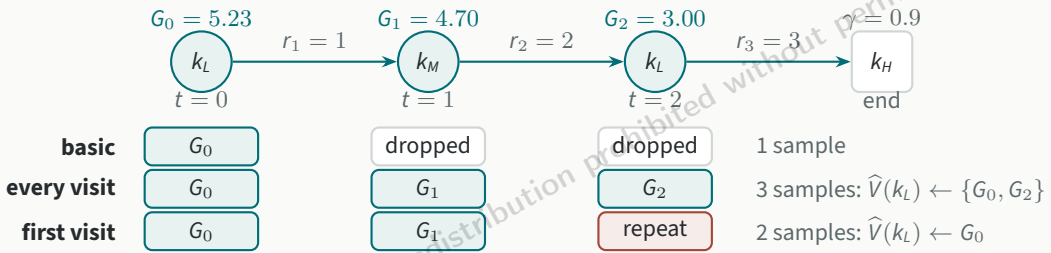
One Path, Many Samples

Reuse every visit a path makes, and pay for it in coverage



© Zhigang Feng Redistribution prohibited without permission

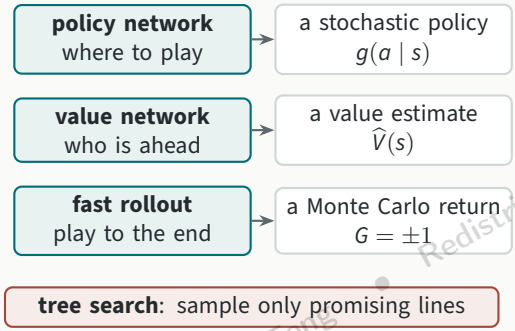
One Path Updates Every Cell It Visits, and the Price



© Zhigang Feng Redistribution prohibited without permission

- The rest of a path samples every state it passes through: one path updates many cells.
- **First-visit**: a state counts once per path; its estimate is the mean of its $C(s)$ returns, the **visit count**. This session uses it.
- The price: a cell gets data only where the rule walks. Harmless for valuing a fixed rule, decisive for improving one: nothing is learned about actions it never takes.
- **A second price: a whole return is noisy and arrives late. Deck 3.3 changes both.**

AlphaGo: a Policy, a Value, and Monte Carlo Returns



- Go: about 2.1×10^{170} legal positions, some 250 moves per turn. Ten moves ahead is $250^{10} \approx 10^{24}$ lines.
- The rules are known. A full backup is still impossible.
- Policy network: trained on 30 million human positions, then improved by self-play.
- **A policy, a value and returns are this session's objects; tree search is not. None enumerates the positions.**

© Zhigang Feng · Redistribution prohibited without permission

Source: Positions: J. Tromp, 2016; AlphaGo: D. Silver et al., 2016.

Session 3.2

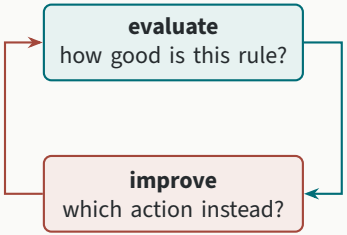
From Evaluation to Control

Improve the rule from its estimated values, and find out what estimation breaks



© Zhigang Feng Redistribution prohibited without permission

Evaluation Was the Easy Half. Now Improve

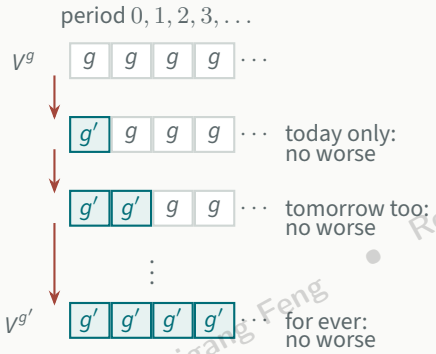


this deck so far:
the top box only

- Everything so far values a *fixed* rule. Finding a better rule from the estimates is **control**.
- The recipe is policy iteration, value the rule then improve it, now on estimated values.
- Any alternation of the two, each possibly incomplete, is **generalised policy iteration**.
- **Next: improvement is safe on exact values; estimates break it (p. 12–14).**

© Zhigang Feng
Redistribution prohibited without permission

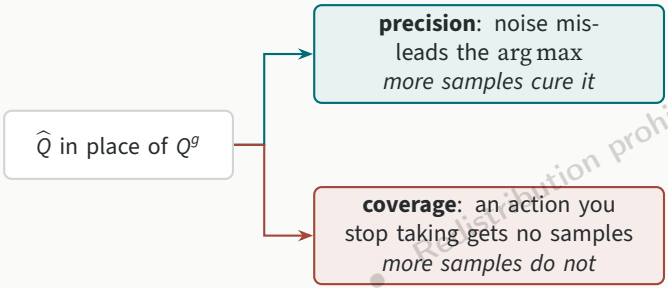
Improving the Rule Cannot Hurt: the One-Shot Deviation Principle



- **Deviate for one period**: take $g'(s)$ today, then follow g. If that is no worse at every state, $Q^g(s, g'(s)) \geq V^g(s)$, the switch cannot hurt.
- **Tomorrow is another today**, wherever g' has led: deviate again. Each row is no worse; with discounting, $V^{g'} \geq V^g$.
- The **one-shot deviation principle**: if no one-period deviation improves a rule, it is optimal.
- **It needs the exact Q^g; Monte Carlo hands us an estimate.**

© Zhigang Feng
Redistribution of materials without permission

Estimation Breaks the Improvement Theorem — Two Ways



© Zhenning Yang. Reproduction prohibited without permission

- The theorem promised: greedy on the *exact* Q^g is never worse. With $\widehat{Q}$ it fails in two ways, needing different cures.
- **One creates mistakes; the other can make them permanent.** A learner that sees only where its own rule goes meets both. The next two pages put numbers on each.

(1) Precision: When Noise Wins the $\arg\max$

- Each cell's estimate is a sample mean with an error of size $\sigma_G/\sqrt{C(s,a)}$: $\widehat{Q}(s,a) = Q^g(s,a) + e_a$. The luckiest estimate wins the $\arg\max$: the **winner's curse** (Deck 3.3's maximisation bias).
- Best action a^*, a rival a, gap $\Delta = Q^g(s,a^*) - Q^g(s,a)$: greedy errs when $e_a - e_{a^*} > \Delta$. With C returns a cell and Φ the normal cdf,

$$P(\text{wrong pick}) \approx \Phi\big(-\Delta\sqrt{C}\,/\,\sigma_G\sqrt{2}\,\big).$$

- One ratio decides, the gap in units of the noise. Ranking right 19 times in 20 takes $C \approx 5.4\,\sigma_G^2/\Delta^2$ returns: halve the gap, four times the data.
- Near the best k' the value is flat, $Q^g(s,a) \approx Q^g(s,a^*) - \frac{1}{2}|Q_{aa}|(a-a^*)^2$, so gaps are small: neighbours are the hardest to rank and the cheapest to confuse. The expected loss $\Delta \cdot P(\text{wrong})$ is at most $0.24\,\sigma_G/\sqrt{C}$, whatever the gap.
- **Small gaps and few samples: the $\arg\max$ picks noise. The error, $\Phi(-\Delta\sqrt{C}/\sigma_G\sqrt{2})$, fades as the count grows.**

© Zhiguo Feng. Unauthorized distribution prohibited without permission

(2) Coverage: the Actions You Stop Taking

- A cell moves only when visited, $\widehat{Q}(s,a) \leftarrow \widehat{Q}(s,a) + \big(G - \widehat{Q}(s,a)\big)/C(s,a)$; its error $\sigma_G/\sqrt{C(s,a)}$ falls only if $C(s,a) \to \infty$.
- Greedy takes a only while $\widehat{Q}(s,a)$ is the row's maximum. Let the best action a^* draw an unlucky first return, $G_1 < \widehat{Q}(s,a')$ for a rival a', which has positive probability: $C(s,a^*)$ stays at 1 and, whatever the number of paths,

$$Q^g(s,a^*) - \widehat{Q}(s,a^*) = Q^g(s,a^*) - G_1 \quad \text{for ever.}$$

- Identification needs **overlap**: $E[G \mid s,a]$ is estimable only if the data's rule takes a at s with positive probability; greedy sets $P(a \mid s) = 0$ for all but one action.
- Rothschild's firm (a **multi-armed bandit**: one state, several prices, unknown demand) never learns a price it stops charging, and can settle on the wrong one for ever.
- **The cure: a rule with $g(a \mid s) \geq \epsilon/|\mathcal{A}(s)|$ visits every cell again and again, so $\widehat{Q} \to Q^g$. Precision errors fade as $\sigma_G/\sqrt{C}$; coverage errors stay, unless every action keeps a positive probability.**

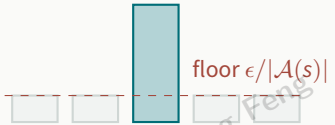
© Zhuge Ting — redistribution prohibited without permission

Two Cures for Coverage: a Reset Button, or a Trembling Hand

exploring starts: a reset button



ϵ-**greedy**: a trembling hand



every k' keeps a floor; the best-looking gets the rest

- **Exploring starts**: a random (s_0, a_0), then g. A reset button only a simulator has.
- ϵ-**greedy**: mostly the best-looking action, with probability ϵ any one. No reset.
- A state visited again and again has every action tried again and again: coverage restored.
- The action is k': a tremble moves the economy to states the rule alone would not visit.
- **Two prices: you learn the value of a rule that trembles, and only where it goes.**

© Zhiyang Feng · Redistribution prohibited without permission

ϵ-Greedy Monte Carlo Control

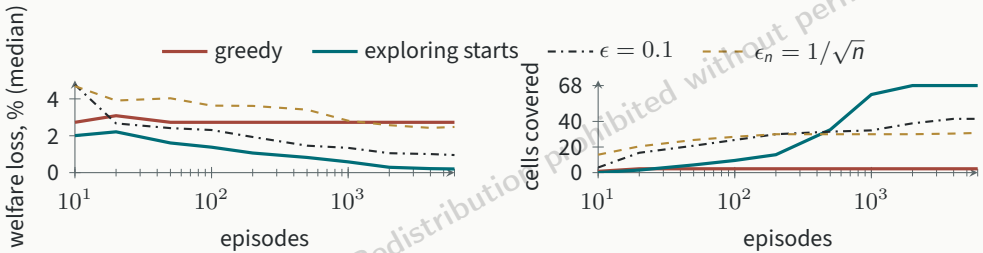
First-visit Monte Carlo control, on the rule's own data

1 initialise $\widehat{Q}$, counts $C \leftarrow 0$; choose a start s_0 (it may be fixed) and a schedule ϵ_i
2 **for** episode $i = 1, \ldots, N$:
3 from s_0, draw every action from g_{ϵ_i} built on the current $\widehat{Q}$; simulate T periods
4 $G \leftarrow 0$; **for** $t = T-1, \ldots, 0$: $G \leftarrow r_{t+1} + \gamma G$
5 first visit of $x = (s_t, a_t)$, first half only: $C(x) \leftarrow C(x) + 1$, $\widehat{Q}(x) \leftarrow \widehat{Q}(x) + \big(G - \widehat{Q}(x)\big)/C(x)$
6 **return** $\widehat{Q}$ and the greedy rule $g(s) = \arg\max_a \widehat{Q}(s, a)$ for deployment

- With exploring starts instead, line 3 draws (s_0, a_0) at random and then follows the greedy rule.
- The rule that collects the data trembles. The rule deployed at the end does not.
- **Trembling, the rule improves to the best trembling rule; to reach g^* the trembles must fade (Deck 3.3). Line 5 is a running mean: Deck 3.3 starts from that line.**

© Zhigang Feng Redistribution prohibited without permission

Measured on the Whole Model: Greedy, Exploring Starts, ϵ-Greedy



Redistribution prohibited without permission

- After 6,000 episodes greedy covers three cells (ten samples or more) and loses most; exploring starts covers all 68 and loses least; $\epsilon = 0.1$ covers 42, in between; $\epsilon_n = 1/\sqrt{n}$ has the right limit but explores too little (0.1 by episode 100).
- Loss: the welfare loss in % of consumption, Deck 3.1's grade, a plain mean over the 14 states.

Source: Recorded runs of the book's own scripts: medians of 20 seeds, one path per cell to start; 7×2 model, $\gamma = 0.95$.

Session 3.2

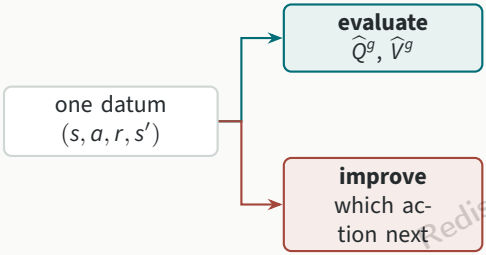
Who Generates the Data

What a datum is, what it is for, and whose value you end up with



© Zhigang Feng · Redistribution prohibited without permission

What the Data Are, and What They Are For



- A datum is one tuple doing two jobs; every method this session chooses how to do each.
- On-policy or off-policy is one question about that pair: **is the rule that generated the tuple the rule being valued?**
- The economist's version: a new Taylor rule, valued on the Fed's post-1979 data.
- **Ask of any result: which rule produced these data, and whose value is being reported?**

© Zhigang Feng Redistribution prohibited without permission

Importance Sampling: Reweighting Someone Else's Data

- **The question.** The paths were walked by the trembling rule b. Can they value the greedy rule g instead?
- Yes, by inverse probability weighting: weight each return by how much likelier g makes its path, the **importance ratio** $\rho = \prod_j g(a_j \mid s_j)/b(a_j \mid s_j)$.
- The shocks have the same probabilities under both rules and cancel: ρ needs no P_z.
- A greedy g takes one action per state, so one tremble zeroes a path's weight and none inflates it: on long paths most weights are zero, a few huge.
- It needs overlap: b must keep trying every action g might take.
- **The ratio is a product over the whole path, and that is the problem: Q-learning (Deck 3.3) is off-policy one step at a time, with no product.**

© Jinsong Pang. Non-distribution prohibited without permission

On-Policy and Off-Policy, in the Algorithm

Monte Carlo from ϵ-greedy data, two ways

1 initialise $\widehat{Q}$; the **behaviour rule** $b = g_\epsilon$, built on $\widehat{Q}$, collects the data
2 **for** each episode: simulate T periods, every action drawn from b
3 $G \leftarrow 0$; **for** $t = T-1, \ldots, 0$: $G \leftarrow r_{t+1} + \gamma G$; at first visits only:
4 **on-policy:** $\widehat{Q}(s_t, a_t) \leftarrow$ running mean of G (the value of b itself)
5 **off-policy:** $\rho \leftarrow \prod_{j>t} g(a_j \mid s_j)/b(a_j \mid s_j)$, with g greedy in $\widehat{Q}$
6 $\widehat{Q}(s_t, a_t) \leftarrow$ mean of G weighted by ρ (the value of g)
7 **return** the greedy rule in $\widehat{Q}$

- The same paths, two **target rules**: line 4 values the rule that collected the data, trembles included (**on-policy**); lines 5–6 reweight by the previous page's ρ and value the greedy rule g (**off-policy**).
- This deck's two controls were on-policy. **Lines 4 and 5–6 are the whole difference.**

© Shuguang Zhang
Redistribution prohibited without permission

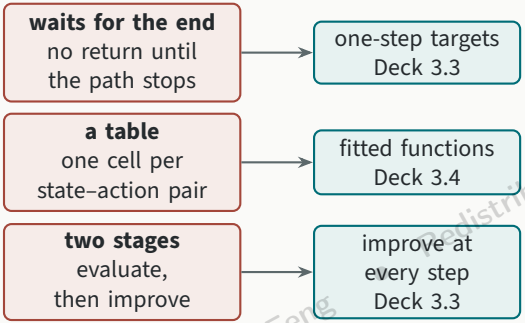
On or Off: the Trade-off

	on-policy: ϵ-greedy MC control	**off-policy**: MC with importance sampling
whose value	the trembling rule you ran	the greedy rule, from the trembling data
each path	counts as it is	reweighted: one tremble zeroes it, none inflates it
noise	an average's	a product's, growing with the path
to reach g^*	trembles must fade, but slowly	any rule still trying every action of g
old data	no: the current rule's only	yes, with known probabilities covering g

- **On-policy: an accurate value of a slightly wrong rule, the trembling one. Off-policy: a noisy value of the right rule, from any rule's data.**
- A new Taylor rule on old-regime data is off-policy; huge weights mean thin overlap.
- Reweighting assumes only actions differ; Lucas: the economy's responses change too.
- Trembles are free in a simulator, costly in an economy: real data push off-policy.
- Whole returns make off-policy dear; one-step targets make it cheap (Deck 3.3).

© Zhiqiang Feng. Redistribution prohibited without permission

What Tabular Monte Carlo Cannot Do



© Zhigang Feng
Redistribution prohibited without permission

- Three limits, each naming what comes next: waiting for the path's end; a table in high dimension; evaluation and improvement in two stages.
- The third is a way of teaching the method, not a requirement: SARSA and Q-learning improve at every step.
- Beyond the three limits, one cost: the variance of whole-path returns (p. 7); one-step targets cut it too.
- **Next, Deck 3.3: update after one step, and stop waiting for the path to end.**

Session 3.3

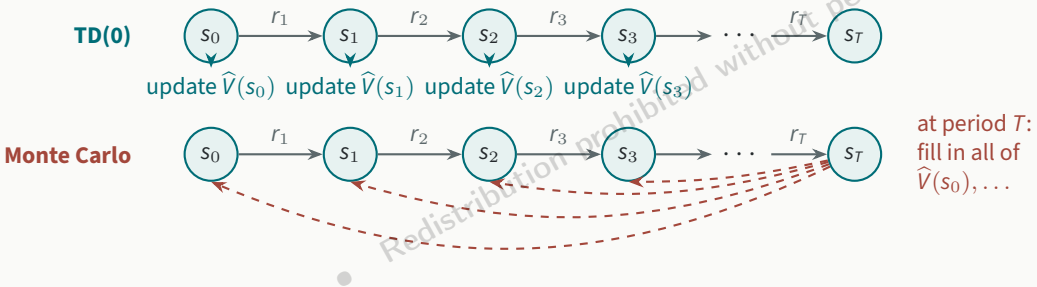
Temporal Difference: TD(0), SARSA, Q-Learning



Zhigang Feng

© Zhigang Feng
Redistribution prohibited without permission

Monte Carlo Waits for the Whole Path. Temporal Difference Does Not



Redistribution prohibited without permission

© Zhaonan Feng

- Deck 3.2 averaged whole returns, $T = 300$: each sample waits T periods and carries every shock on the way.
- **This deck takes $T = 1$: one real reward, and the current estimate for everything after.**

Session 3.3

Evaluation, One Step at a Time

The incremental mean, TD(0), and everything between one step and the whole path



© Zhigang Feng · Redistribution prohibited without permission

The Sample Mean, Written One Observation at a Time

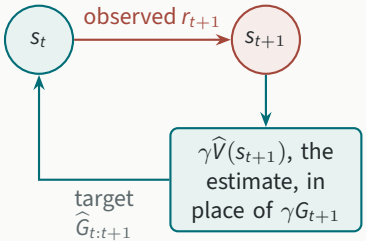
- **The question.** Deck 3.2 averaged returns. How is an average kept up to date as the draws arrive?
- The mean of $n+1$ draws is the old mean plus a correction:

$$\bar{X}_{n+1} = \bar{X}_n + \frac{1}{n+1}\left(X_{n+1} - \bar{X}_n\right).$$

  New estimate $=$ old estimate $+$ step size $\times$ surprise: adaptive expectations with gain $1/(n+1)$.
- Any **step size** η_n will do if it shrinks, but not too fast (the Robbins–Monro conditions): the estimate then settles on the mean.
- A constant η never settles: the estimate hovers around the mean, forgetting old draws at rate $1-\eta$.
- **Forgetting is a virtue when the target itself moves, as it will once the rule is being improved.**

© Zhiguang Feng
Redistribution prohibited without permission

TD(0): One Real Reward, the Rest on Credit



- **The question.** A return waits for the whole path. Can one period teach something?
- Yes: borrow the estimate for the unseen rest. The **TD target** $\widehat{G}_{t:t+1} = r_{t+1} + \gamma \widehat{V}(s_{t+1})$ is the Bellman equation on one draw.
- Nudge the stored value toward it, $\widehat{V}(s_t) \leftarrow \widehat{V}(s_t) + \eta\, \delta_t$, where the **TD error** $\delta_t = \widehat{G}_{t:t+1} - \widehat{V}(s_t)$ is the miss.
- Leaning on your own estimate is **bootstrapping**: one real reward, the rest on credit.
- **Against DP: no sum over z'. Against Monte Carlo: no waiting.**

© Zhigang Feng
Redistribution prohibited without permission

TD(0) Policy Evaluation

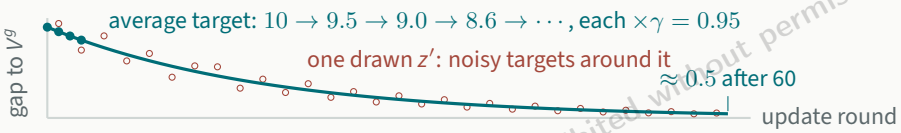
TD(0): evaluate a given policy g along one simulated path

initialise $\widehat{V}(s)$; visit counts $C(s) \leftarrow 0$; $s \leftarrow s_0$
for $t = 0, 1, 2, \ldots$ up to the step budget:
  draw $a \sim g(\cdot \mid s)$; the simulator returns r and s'
  $C(s) \leftarrow C(s) + 1$
  $\delta \leftarrow r + \gamma \widehat{V}(s') - \widehat{V}(s)$
  $\widehat{V}(s) \leftarrow \widehat{V}(s) + \eta\, \delta$ η constant, or shrinking with $C(s)$
  $s \leftarrow s'$

- It converges to V^g if every state keeps being visited and the step shrinks at the right pace; nothing in that says how fast.
- The plain average, $\eta = 1/C(s)$, qualifies and is unusably slow (p. 12).
- **No episodes are needed: the path just runs. Cutting it is not ending it.**

© Thierry Post · Redistribution prohibited without permission

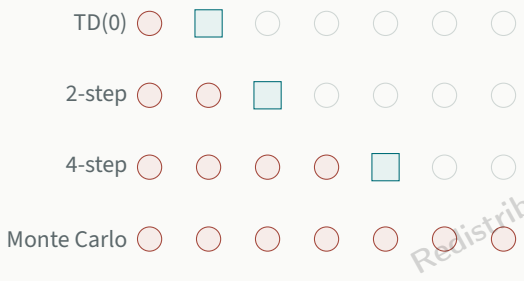
Why Tabular TD(0) Converges: the Target Contracts



- **The question.** The target $r + \gamma\widehat{V}(s')$ leans on a guess. Why does that not go astray?
- **The target is less wrong than its guess.** r is real; only tomorrow is guessed, and it is discounted. A guess off by 10 everywhere gives a target off by $0.95 \times 10 = 9.5$: on average, one step of value iteration for g.
- **The noise averages out.** One drawn z' scatters the target around that average; a shrinking step, $\eta = C(s)^{-0.6}$, lets no single draw move the estimate far.
- **Every state keeps being visited.** A cell is fixed only when the path passes through it; the trembles see to that.
- **TD is noisy value iteration, one state at a time. Cells share nothing, so fixing one never spoils another; functions that share lose the guarantee (Deck 3.4).**

© Zhang Fengyi. Redistribution prohibited without permission

n-Step Returns: Between One Step and the Whole Path



© Zhigang Feng
Redistribution prohibited without permission

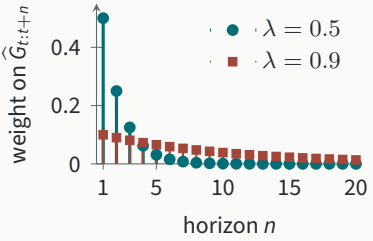
- **The question.** How many real periods before the estimate? In the lab, five beat one with the branches summed, not with one drawn.
- The ***n*-step return**: n real rewards, then the estimate,

$$\widehat{G}_{t:t+n} = \sum_{j<n} \gamma^j r_{t+j+1} + \gamma^n \widehat{V}(s_{t+n}).$$

- $n = 1$ is TD(0); large n is Monte Carlo. Larger n: less bias, more variance (why: p. 10).
- **And the update must wait n periods.**

Source: Session 3 lab, block B.

TD(λ): Average Every Horizon, Credit the Recent States



- **The question**: n-step returns make us pick one n. Why not average every horizon? **TD(λ)** does, with weights that fade:

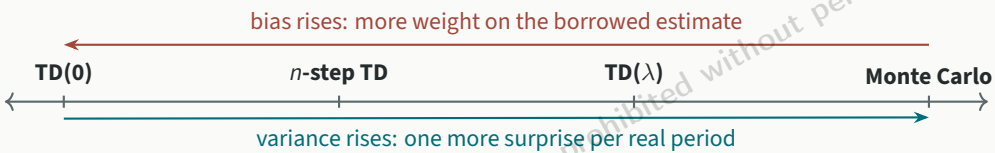
$$\widehat{G}_t^{\lambda} = (1-\lambda)\sum_{n\geq 1}\lambda^{n-1}\widehat{G}_{t:t+n}.$$

$\lambda = 0$: TD(0)'s target. $\lambda \to 1$: Monte Carlo's whole return.

- It can be computed as the path runs: when today's surprise δ_t arrives, credit it to the states visited lately, a state k periods back taking the share $(\gamma\lambda)^k$.
- The **eligibility trace** is the bookkeeping that remembers those shares; summed over the path, the updates are the same average over horizons.
- **TD(λ) charges today's surprise to the states that led here, recent ones most: credit assignment, which choice earned the reward.**

© Lilianna Pérez Romero. Redistribution prohibited without permission

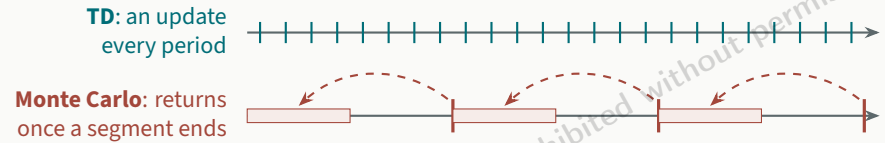
One Dial: Bias against Variance



© Zhang Fang. Redistribution prohibited without permission

- **The question.** Why does a longer target trade bias for variance?
- A target's error has two parts. Each real period adds one surprise, a shock or a tremble, mean zero and fresh every period: that is noise, and n periods add n of it.
- The borrowed estimate $\gamma^n \widehat{V}(s_{t+n})$ is the only part that can be wrong on average: that is bias, all of it in TD(0), none in Monte Carlo, discounted by γ^n in between.
- **The bias shrinks as $\widehat{V}$ learns; the variance does not. Early, lean on real rewards; later, on the estimate.**

TD against Monte Carlo on One Long Path: Bias Early, Variance Late



© Stefano Fenig • Redistribution prohibited without permission

- **The set-up.** One fixed exploring rule, one long path, no resets, as an economy produces data. TD uses each transition at once; Monte Carlo cuts the path into segments and waits.
- **Early**, Monte Carlo wins: its returns hold no guess, so its first segments set the right level. TD's targets are mostly the starting guess, fading one period back per update.
- **Later**, TD wins: the borrowed error is small, and only noise remains: one surprise in TD's target, a whole segment's in Monte Carlo's.
- How fast the guess fades is the step size's doing (p. 12).
- **Bias fades as the estimate learns; variance does not. That decides who leads.**

Target and Step Size Cannot Be Judged Apart

- **The question.** TD converges with the plain average $\eta = 1/C(s)$. Why is that step unusably slow?
- The step says how much each target counts. The plain average counts every target equally, which suits Monte Carlo: its returns are equally good from the first.
- TD's early targets rest on the starting guess. Counted like the later, better ones, the guess fades so slowly that halving its error takes more visits than any budget allows.
- A step that shrinks more slowly than $1/C(s)$ lets go of early targets while still settling: far faster, and still convergent.
- The start matters to TD alone. Steady-state utility for ever is a sensible guess; under the plain average, only a start near the truth ends near it.
- **With a bootstrapped target, choose the target, the step size and the start together.**

© Zhirang Feng · Redistribution prohibited without permission

Session 3.3

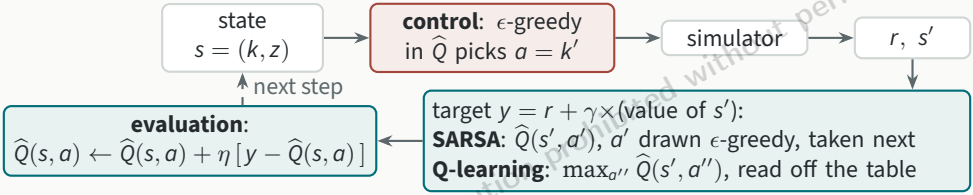
Control: SARSA and Q-Learning

Improve the rule at every step, and ask whose value is being learned



© Zhigang Feng · Redistribution prohibited without permission

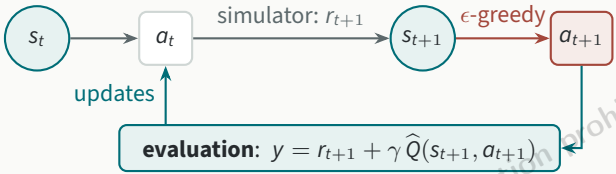
From Evaluation to Control: Improve at Every Step



© Zhirang Feng · Redistribution prohibited without permission

- **Control** improves the rule (Deck 3.2): act ϵ-**greedily** in the latest $\widehat{Q}$, the best-looking k' usually, a random feasible one now and then.
- **Evaluation**: one TD update of $\widehat{Q}(s, a)$, the value of taking a in s, toward the target y: p. 5's one-step target with $\widehat{Q}$ for $\widehat{V}$.
- Both see the same r and s'; the targets differ only after an exploring draw.
- **SARSA values tomorrow by the rule it follows, Q-learning by the greedy rule.**

SARSA, Step by Step



control: drawn from the rule in the current $\widehat{Q}$, and really taken next

© Zhigang Feng
No distribution prohibited without permission

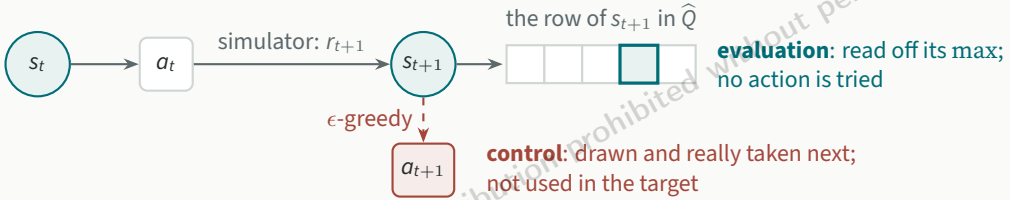
- In s_t, take a_t, chosen a period earlier; the simulator returns r_{t+1} and s_{t+1}.
- **Control**: in s_{t+1}, draw a_{t+1} ϵ-greedily in the current $\widehat{Q}$. It is the action the economy will really take next.
- **Evaluation**: $\widehat{Q}(s_t, a_t) \leftarrow \widehat{Q}(s_t, a_t) + \eta\left[r_{t+1} + \gamma\,\widehat{Q}(s_{t+1}, a_{t+1}) - \widehat{Q}(s_t, a_t)\right]$.
- Move on, $(s_t, a_t) \leftarrow (s_{t+1}, a_{t+1})$. Each step uses (s, a, r, s', a'): hence the name, **SARSA**.
- **One rule does both jobs: it chooses the actions, and it is the rule being valued.**

What SARSA Learns, and Where Its Control Ends

- **The question.** SARSA improves the rule at every step. Whose value does it learn, and does it reach g^*?
- The value of the rule making the data, trembles included (**on-policy**). A trembling saver wastes capital tomorrow, so it learns to save a little less than g^* would.
- With ϵ fixed it settles on the best **ϵ-soft** rule, one keeping every action at probability $\geq \epsilon/|\mathcal{A}(s)|$: the best way to tremble, not g^*.
- **GLIE**, greedy in the limit with infinite exploration: let ϵ fall to zero slowly, every pair still tried; then SARSA reaches g^*.
- **Expected SARSA** averages the target over a' instead of drawing it: same mean, less noise.
- **SARSA learns the value of exploring, and improves the rule it follows.**

© Zhigang Feng · Redistribution prohibited without permission

Q-Learning, Step by Step



© Zhirang Feng · Redistribution prohibited without permission

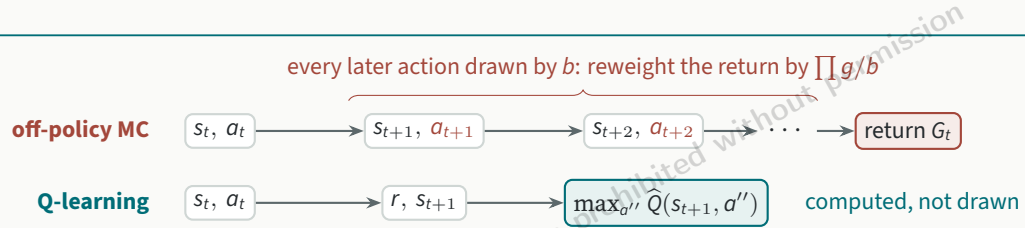
- The transition is SARSA's: in s_t take a_t; the simulator returns r_{t+1} and s_{t+1}.
- **Evaluation**: $\widehat{Q}(s_t, a_t) \leftarrow \widehat{Q}(s_t, a_t) + \eta \left[r_{t+1} + \gamma \max_{a'' \in \mathcal{A}(s_{t+1})} \widehat{Q}(s_{t+1}, a'') - \widehat{Q}(s_t, a_t) \right]$. No action is tried: the max reads numbers already in the table.
- **Control**: in s_{t+1}, draw a_{t+1} ϵ-greedily and take it. It drives the path; the target ignores it.
- **Two rules, two jobs: the ϵ-greedy rule acts, and the greedy rule is valued.**

What Q-Learning Learns: Q^*, from Exploring Data

- **The question.** Q-learning's target reads a $\max$ off the table instead of the action taken. What does that buy?
- Value function iteration, done one sampled transition at a time: with the expectation exact and $\eta = 1$, the update is Deck 3.1's value iteration. The improvement sits inside the target.
- So it values the greedy **target rule** from the data of the ϵ-greedy **behaviour rule**: **off-policy**. A fixed $\epsilon > 0$ suffices; deploy the greedy rule, exploring was for learning.
- Three costs remain: cells rarely visited stay wrong; the $\max$ over noisy estimates is biased up (p. 24); welfare while learning is the trembling rule's.
- **Off-policy buys the right target, not the right data.**

© Zhiguo Feng Redistribution prohibited without permission

Why Q-Learning Needs No Importance Ratio



- **The question.** Q-learning values the greedy rule g from the exploring rule b's data, yet Deck 3.2's off-policy Monte Carlo had to reweight each return by $\prod g/b$. Why not here?
- Who chose each part of the target? a_t is given, the update is for that cell; r and s_{t+1} are the economy's under any rule; the $\max$ is computed for g.
- Nothing b drew after a_t enters the target, so there is nothing to reweight. A Monte Carlo return holds every later action, each drawn by b: hence the product.
- **One step, no ratio: b only decides which cells get data. A target holding more of the walked path would bring the ratio back.**

© Zhiyang Feng · Reproduction prohibited without permission

One Algorithm, One Line of Difference

TD control: SARSA and Q-learning

1 initialise $\widehat{Q}$ on feasible cells, counts $C \leftarrow 0$; $s \leftarrow s_0$; choose a, ϵ-greedy in $\widehat{Q}$ — **control**

2 **for** $t = 0, 1, 2, \ldots$ up to the step budget:

3 take a; the simulator returns r and s'

4 choose a' in $\mathcal{A}(s')$, ϵ-greedy in $\widehat{Q}$: the action taken next — **control**

5 **SARSA:** $y \leftarrow r + \gamma\, \widehat{Q}(s', a')$ **Q-learning:** $y \leftarrow r + \gamma \max_{a'' \in \mathcal{A}(s')} \widehat{Q}(s', a'')$ — **evaluation**

6 $C(s,a) \leftarrow C(s,a) + 1$; $\widehat{Q}(s,a) \leftarrow \widehat{Q}(s,a) + C(s,a)^{-0.6}[\, y - \widehat{Q}(s,a)\,]$ — **evaluation**

7 $(s, a) \leftarrow (s', a')$

- **Control** (lines 1, 4) is the same in both: ϵ-greedy in the current $\widehat{Q}$, which line 6 updates.
- **Evaluation** (lines 5–6): SARSA values the a' that line 4 drew, so one rule acts and is valued (on-policy); Q-learning values the greedy choice, and a' only acts (off-policy).
- **Line 5 is the whole difference. Deploy $g(s) = \arg\max_a \widehat{Q}(s,a)$ in both.**

Source: Session 3 lab, block E.

© Eduardo Fe… Reproduction prohibited without permission

Session 3.3

Using Samples Well

Q-learning reaches Q^* in the limit. On a budget of steps, where do samples go to waste, and how is each waste cut?



© Zhigang Feng · Redistribution prohibited without permission

Q-Learning on a Budget: Four Places Samples Go to Waste

2 how much it counts: the step size (p. 12)

3 a max over noisy estimates (p. 24)

$$\widehat{Q}(s,a) \leftarrow \widehat{Q}(s,a) + \eta_{C(s,a)} \left[r + \gamma \max_{a''} \widehat{Q}(s',a'') - \widehat{Q}(s,a) \right]$$

1 which (s,a) gets the sample: the behaviour rule (p. 23)

4 one real period, then the estimate (p. 8–10)

© Zhening Wang · Redistribution prohibited without permission

- **The question.** Convergence asks for every pair visited for ever and a shrinking step, but not *how many* steps. On a budget, each part of the update can waste samples.
- Two of the four were met on the evaluation side: the step size, and how far the target looks. The other two are about the data and the max: next.
- **Every fix changes where samples come from, or how the target uses them.**

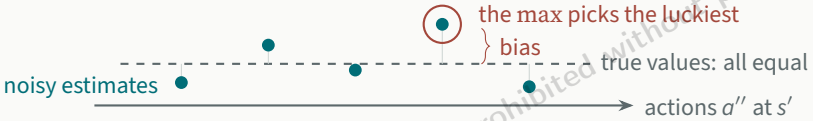
Which Cells Get Data: Explore Where It Can Pay



© Zheng Tang Redistribution prohibited without permission

- **The problem.** Q-learning learns Q^* for any fixed $\epsilon > 0$, but only in cells with data, and ϵ-greedy spreads its trials evenly: a hopeless k' gets as many as the runner-up.
- **Boltzmann exploration**: a logit choice rule, $g(a \mid s) \propto \exp\{\widehat{Q}(s,a)/\tau\}$ with temperature τ; the worse an action looks, the fewer its trials.
- **Optimistic initial values**: start $\widehat{Q}$ above the values it will learn; an untried action looks good until tried.
- **Pace**: ϵ or τ may fall with the counts, as long as every pair keeps being tried.
- **Explore near the best, and where you have not looked.**

What the Max Adds: Maximisation Bias, and a Second Table



- **The problem.** The target's max is over noisy estimates, and $E[\max \widehat{Q}] \geq \max E[\widehat{Q}]$: the action that looks best is partly the luckiest (Deck 3.2's winner's curse).
- Bootstrapping passes the inflated value back to earlier states. SARSA is not immune: its a' is the greedy action most of the time.
- **The fix**, **double Q-learning**: keep two tables and update one of them, chosen at random, each step. One table picks the winner, the other prices it; act on their sum.
- **Luck that picks an action no longer lifts its value.**

© Zhigang Feng — Not to be distributed without permission

The Backup Spectrum: Two Choices Define an Evaluation Method



- **The question.** Every evaluation in Decks 3.1–3.3 solves $V^g = T^g V^g$. What differs is the **backup**, the target used.
- **Width**: sum every branch, exact but reading P_z, or draw one, no P_z but one draw's noise. DP sums; this deck samples: we pretend not to know P_z.
- **Depth**: lean on the stored $\widehat{V}$ after one period, quick but biased while it is poor, or run to the end, honest but noisy and slow; n, λ, step and start tune it.
- **Every point on the map is the same Bellman equation.**

© Zhigang Feng. Redistribution prohibited without permission

Session 3.4

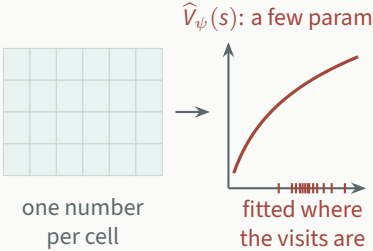
From Tables to Networks: Function Approximation, Policy Gradient, Actor–Critic



Zhigang Feng

© Zhigang Feng
Redistribution prohibited without permission

Where a Table Stops Fitting: Store a Function Instead



- Decks 3.2–3.3 kept a table, one number per cell. **Deck 3.1's two learners store two functions instead: how?**
- In the lab: the rule a sigmoid of a line in $\ln k$, shifted for z, **three numbers**; the value a quadratic in $\ln k$, its level and slope shifted for z, **five numbers**.
- **Function approximation**: far fewer parameters than states, fitted where samples fall; a table is the indicator case.
- **Two routes: a basis an economist picks, then features a network learns. Then the rule carries the $\max$, in steps.**

Source: Session 3 lab, block B.

© Zhigang Feng
Redistribution prohibited without permission

Session 3.4

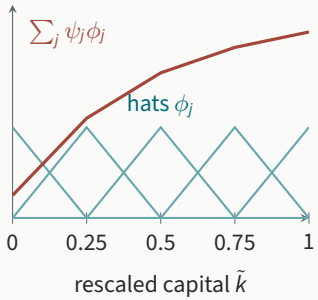
Route One: Linear in Parameters

A basis the economist picks: what it is, what dimension costs, what it buys and what it costs



© Zhigang Feng · Redistribution prohibited without permission

Linear in the Parameters, Not in the State



- **Yes: linear in the weights ψ, not in k.** $\widehat{V}_\psi(s) = \sum_j \psi_j \, \phi_j(s)$, each **basis function** ϕ_j a fixed curve; a table is the indicator basis.
- The gradient is $\phi(s)$: one update moves every state sharing a feature (p. 5).
- On the grid: $\phi(k, z) = (1, \tilde{k}, \tilde{k}^2, \mathbf{1}\{z = 1.1\}, \tilde{k}\,\mathbf{1}\{z = 1.1\})$, $\tilde{k}$ capital rescaled to $[0, 1]$.
- A smooth, concave value: polynomials in $\ln k$. A kink, as at a borrowing limit: local pieces.
- **The economist chooses the ϕ_j, so must guess the value's shape. Next: what that buys, what it costs.**

© Zhigang Feng
Redistribution prohibited without permission

Route One: What It Buys, What It Costs, and Why Route Two

	a basis you pick: this route	**features learned**: Route Two
you choose	the curves ϕ_j; they must contain the value's shape ($\ln k$ terms for log utility)	an architecture; its hidden layers are the features
one update	$\psi \leftarrow \psi + \eta\, \delta_t\, \phi(s_t)$, δ_t the TD error, the target held fixed	a gradient step; a frozen copy holds the target (p. 8)
many state variables	all products of the 1-D curves: m^d weights; sparse grids thin them	the first layer grows with d, not m^d: Deck 3.1's First Answer
guaranteed	on the rule's own data: error within a constant of the best fit's	no general result
can go wrong	the **deadly triad**: a fitted value, a self-made target and another rule's data can run away together	the same; Q-learning on a network has all three

- **Why Route Two: a network learns the features a basis must guess, model by model, and can take a histogram as one input (Deck 3.5). The price is the guarantee.**

© Zhuangyuan Fan Redistribution prohibited without permission

Session 3.4

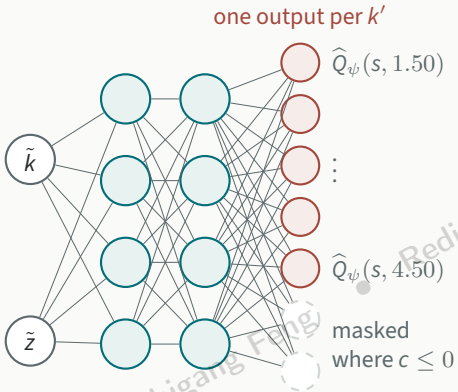
Route Two: Learn the Features

A network for Q; then a rule of its own, which carries the $\max$ in gradient steps



© Zhigang Feng · Redistribution prohibited without permission

Let the Features Be Learned: a Network for Q



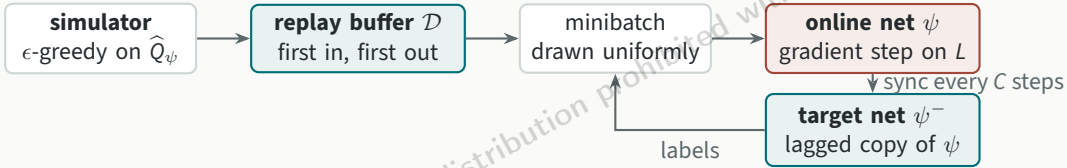
hidden layers: the learned features

- **Why one output per k'**: Q-learning's target needs $\max_{a'} \widehat{Q}(s', a')$; with an output per k', one pass returns Deck 3.3's whole row.
- With k' as an input, each candidate costs a pass and the $\max$ is a search. So DQN suits a few discrete actions; a share has no row (p. 9).
- The hidden layers *are* the basis, now trained on the data instead of guessed. Weights ψ; θ is kept for the rule.
- The triad of p. 5 is complete: a fitted value, a self-made target, exploring data. **DQN adds two devices that hold the regression still.**

© Zhigang Feng. Redistribution prohibited without permission

DQN: a Target Network and a Replay Buffer

- **The question.** Each step on ψ moves $\widehat{Q}$ everywhere, the label included, and successive transitions are alike. What holds the regression still?



$$L(\psi) = E_{(s,a,r,s')\sim\mathcal{D}}\left[\left(r + \gamma \max_{a'\in\mathcal{A}(s')} \widehat{Q}_{\psi^-}(s',a') - \widehat{Q}_\psi(s,a)\right)^2\right].$$

- **Target network**: a frozen copy ψ^- holds the labels still for C steps.
- **Experience replay**: minibatches from stored transitions mix times and places.
- **Neither removes the triad: the loss and the rule still swing. Keep the best iterate, found by evaluating, not the last.**

Source: Session 3 lab, block F.

© Zhigang Feng Redistribution prohibited without permission

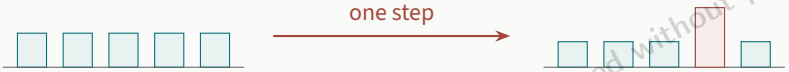
No Row to Maximise: Improve the Rule Itself, from Samples

- **The question.** DQN maximises over a row; a saving share has none, and $\arg\max_a \widehat{Q}_\psi$ can jump on a small change in ψ. How else can a rule improve?
- **The answer: make the rule a distribution** $g_\theta(a \mid s)$, smooth in θ. It tries actions; those that did better than expected become likelier.
- Why random, when p. 15 moves a share along a derivative? Without the model, a better action shows only if it is tried: Deck 3.2's coverage.
- And the derivative then runs through the rule's own odds, which we know, not through the environment, which we do not.
- On the grid: a probability on each feasible k'. For a share: p. 2's sigmoid share, with a tremble added.
- **Q-learning fits a value, then maximises. A policy gradient climbs V^{g_θ} directly: the rule carries the $\max$, one step at a time.**

© Zheng Jiang · Redistribution prohibited without permission

REINFORCE: Try, Score, Reinforce

$g_\theta(k' \mid s)$ **over the feasible** k'**: uniform** **after one path: the** k' **taken (red) did better than expected**

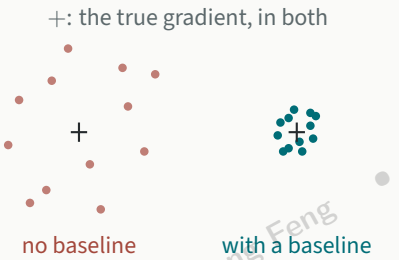


- **Try**: simulate T periods under the current random rule g_θ; its randomness explores.
- **Score**: the **score** $\nabla_\theta \ln g_\theta(a_t \mid s_t)$ points where a_t becomes likelier: $+(1-g)$ on the action taken, $-g$ on each other.
- **Reinforce**: weight each score by what followed, $G_t = \sum_{j\geq 0} \gamma^j r_{t+j+1}$ (credit assignment, Deck 3.3), less a **baseline** $b(s_t)$ of the state alone: $\theta \leftarrow \theta + \eta \sum_t \gamma^t \nabla_\theta \ln g_\theta(a_t \mid s_t)\big(G_t - b(s_t)\big)$.
- On average the step is the exact uphill direction of the rule's lifetime value; nature's odds carry no θ, so only the rule's own odds are differentiated, never the model.
- **One gradient step up the log-likelihood of your own choices, weighted by what followed. No model, no value function.**

© Zhiyang Feng. Redistribution prohibited without permission

REINFORCE with a Baseline

$$\theta \leftarrow \theta + \eta \sum_t \gamma^t \nabla_\theta \ln g_\theta(a_t \mid s_t) \left(G_t - \widehat{V}_\psi(s_t)\right), \qquad \widehat{V}_\psi \text{ moved toward the returns } G_t$$



- **The question.** Without b, every action tried in a good state looks good, lucky or not. Judge against what?
- Against the state's usual payoff $\widehat{V}_\psi(s)$, fitted to the same returns: what is left is better or worse than expected, the **advantage**.
- A function of the state alone leaves the mean step unchanged: a control variate.
- **Same average direction, far less noise. This is the critic's first job (p. 24).**

© Zhigang Feng
Redistribution prohibited without permission

Session 3.4

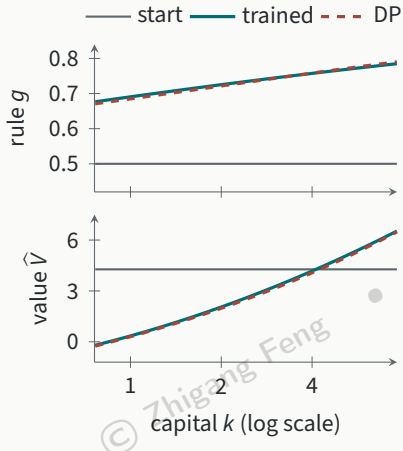
Actor–Critic

Deck 3.1’s two learners, step by step: with the model in hand, then without it, then on networks



© Zhigang Feng · Redistribution prohibited without permission

Two Functions: a Rule and a Value



- **The question.** Model in hand, can two functions and two steps find DP's rule with no grid, no sweep and no search over actions?
- The **rule** $g(s)$: the saving share at $s = (k, z)$, so $c = (1 - g)\, y > 0$. The **value** $\widehat{V}(s)$: what s is worth under the current rule.
- Each is a function to evaluate and nudge anywhere: Route One's forms, p. 2.
- The rule is deterministic: one share per state.
- **Trained: the rule on DP's curve, the value close to** V^* (why: valuing a fixed rule is Deck 3.1's contraction T^g).

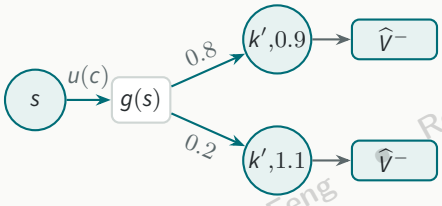
© Zhigang Feng
Redistribution prohibited without permission

Source: Session 3 lab, block B; $z = 0.9$; trained: median of 20 seeds; how they are stored: p. 2.

The Critic's Step: Move the Value Toward a One-Period Backup

$$B(s) = u(c) + \gamma \sum_{z'} P_z(z' \mid z)\, \widehat{V}^{-}(k', z'), \quad c = (1 - g(s))\, y, \quad k' = g(s)\, y$$

$$\widehat{V} \leftarrow \arg\min_{\widehat{V} \in \mathcal{F}} \sum_{i=1}^{N} \left(\widehat{V}(s_i) - B(s_i)\right)^2$$



at $z = 0.9$: both branches weighted

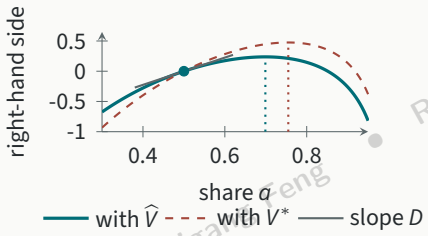
- $B(s)$: Deck 3.1's one-period backup (*Challenge 2: Why Look Further Ahead*) at one state; $\widehat{V}^{-}$: the value as the iteration began, held fixed.
- $\mathcal{F}$: what the critic can store. A table copies B into cells; any other form takes the closest fit (p. 5).
- First a **prefit**: 200 iterations with the rule frozen, so the critic starts by valuing $g \equiv \frac{1}{2}$ right.
- T^{g} **(Deck 3.1, *Two Value Functions*) applied once, exactly, then written into the value.**

© Zhigang Feng
Redistribution prohibited without permission

The Actor's Step: Which Way to Move the Share Is a Derivative

$$\max_g J(g) = \tfrac{1}{N}\sum_i \Big[u\big((1-g(s_i))\,y_i\big) + \gamma \sum_{z'} P_z(z' \mid z_i)\,\widehat{V}\big(g(s_i)\,y_i,\, z'\big)\Big], \quad \widehat{V} \text{ held fixed}$$

$$D(s) = \partial_a[\,\cdot\,]_{a=g(s)} = y\Big[-u'(c) + \gamma \sum_{z'} P_z(z' \mid z)\,\partial_k \widehat{V}(k', z')\Big]$$



- One small step uphill in J; $D(s_i)$ is its slope at s_i (the sigmoid of p. 2 keeps g in $(0, 1)$).
- A **pathwise gradient**: differentiate through the budget constraint, into u and the critic.
- Peak at 0.699: Howard's improvement of the flat rule. DP's: 0.755.
- **Still a $\max$, taken in steps**: one step uphill, not Howard's jump to the peak at 0.699.

Source: Session 3 lab, block B; $k = 3.75$, $z = 0.9$; curves shifted to zero at $a = \frac{1}{2}$.

© Zhigang Feng
Redistribution prohibited without permission

The Whole Algorithm on One Page

Actor–critic with the model in hand, on drawn states

1 initialise $g \equiv \frac{1}{2}$ and $\widehat{V} \equiv u(c^*)/(1-\gamma) = 4.27$ (c^*: steady-state consumption)
2 **prefit**: repeat line 5 with the rule frozen, 200 times
3 **repeat**
4 draw $N = 256$ states, uniform in $\ln k$ over the band and over $z \in \{0.9, 1.1\}$
5 **critic**: $B(s_i) \leftarrow u(c_i) + \gamma \sum_{z'} P_z(z' \mid z_i)\, \widehat{V}^{-}(k_i', z')$; fit $\widehat{V}$ to B at the s_i
6 **actor**: $D(s_i) \leftarrow y_i \big[-u'(c_i) + \gamma \sum_{z'} P_z(z' \mid z_i)\, \partial_k \widehat{V}(k_i', z') \big]$; one step uphill in J
7 **until** the rule stops moving (1000 iterations here); grade g against the key

- Lines 5 and 6 read u, f and P_z: the model, whole. Nothing is drawn but the states.
- Every line is written on the two functions; p. 2 said how the lab stores them.
- Deck 3.1's grade: $0.0025\,\%$ loss over the band after 1000 iterations.
- **Deck 3.1's two questions, each answered once per iteration: evaluate on line 5, improve on line 6.**

Source: Session 3 lab, block B; the grade is the median of 20 seeds.

© Miguel Rojas. Redistribution prohibited without permission

Without the Model: One-Step Actor–Critic

One-step actor–critic, online

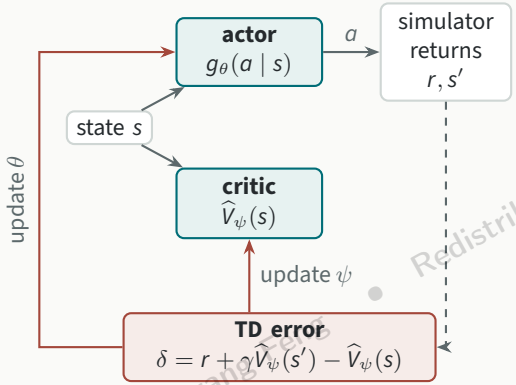
1 initialise θ, ψ; draw s from the start distribution
2 **for** $t = 0, 1, 2, \ldots$ up to the step limit:
3 draw $a \sim g_\theta(\cdot \mid s)$ on the feasible set; the simulator returns r and s'
4 $\delta \leftarrow r + \gamma \widehat{V}_\psi(s') - \widehat{V}_\psi(s)$
5 $\psi \leftarrow \psi + \eta_\psi \, \delta \, \nabla_\psi \widehat{V}_\psi(s)$ **critic**
6 $\theta \leftarrow \theta + \eta_\theta \, \delta \, \nabla_\theta \ln g_\theta(a \mid s)$ **actor**
7 at a reset, redraw s; otherwise $s \leftarrow s'$

- The **critic** evaluates by TD on the actor's own data; the **actor** climbs the score, weighted by δ: better than expected, a grows likelier.
- The rule's forms (p. 9): on the grid a **masked softmax**, $g_\theta(a \mid s) \propto \exp h_\theta(s, a)$ over the feasible k'; for a share, $x \sim N(\mu_\theta(s), \sigma^2)$, $a = \mathrm{sigmoid}(x)$.
- **One TD error does both jobs; the critic must move faster, it is the actor's yardstick.**

Source: Session 3 lab, block G.

© Vincent Pons — Redistribution prohibited without permission

The Data Flow: One TD Error, Two Updates



© Zhigang Feng
Redistribution prohibited without permission

- One number a period, the surprise δ: what happened less what the critic expected. It fixes the value and grades the action.
- The valuer must keep pace with the decider: two step sizes, the critic's the larger.
- The price: the surprise is noisy, and it leans on the critic; a wrong critic misleads the actor.
- **A differentiable model lets the actor differentiate through it instead (p. 15): deep value iteration, next.**

Session 3.4

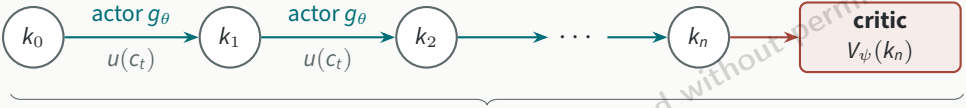
With the Model: Deep Value Iteration

The actor–critic when the model is in hand: both functions networks, both steps n periods deep



© Zhigang Feng · Redistribution prohibited without permission

Deep Value Iteration: the Simplest Actor–Critic's Two Steps, with Networks



- **Deep value iteration is an actor–critic with the model in hand**: the actor steps along a pathwise derivative through n periods; the critic's target is that rollout, detached.
- Session 2's deterministic growth model, its own calibration. Actor g_θ: Session 2's share network. Critic V_ψ: its level started at $u(c^*)/(1-\gamma)$.
- Against p. 17's: no drawn actions, no exploring, a pathwise gradient. Against p. 16's: networks, n periods in both steps, one gradient step a round.
- **The name: at $n = 1$, with the actor at its $\max$, the critic's label is TV_ψ: fitted value iteration. The actor carries the $\max$.**

Source: Session 3 lab, block C.

For Miguel's Use Only
Redistribution prohibited without permission

The Two Steps, Written Out

Deep value iteration, as the lab runs it

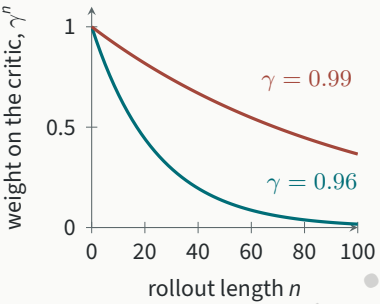
1 initialise the actor near the steady-state share, the critic's level at the steady-state value
2 **repeat** for a fixed number of rounds:
3 draw a fresh batch $k_1, \ldots, k_N$, uniform in $\ln k$ over the band
4 roll the actor n periods: $k_{t+1} = g_\theta(k_t)\, y(k_t)$, $c_t = y(k_t) - k_{t+1}$; form $J_n(k_i)$
5 $\theta \leftarrow \theta + \eta_\theta \nabla_\theta \frac{1}{N} \sum_i J_n(k_i)$ (through the budget constraint and into $V_\psi(k_n)$) **actor**
6 recompute $J_n(k_i)$ under the new θ, **detached**; $\psi \leftarrow \psi - \eta_\psi \nabla_\psi \frac{1}{N} \sum_i \left(V_\psi(k_i) - J_n(k_i)\right)^2$ **critic**

- $y(k)$ is output plus undepreciated capital, the resources. Line 5 is a pathwise gradient: the derivative runs through the model for n periods.
- **Detach**: the label in line 6 passes no gradient, or the critic could cut its loss by moving the label. No frozen copy: the live critic, detached (Deck 3.5's household adds one).
- **No benchmark enters either loss. Actor first, then critic, one gradient step each.**

Source: Session 3 lab, block C, which takes Adam steps.

© Zhaane Penn. Redistribution prohibited without permission

One Step or Many, and Where to Sample

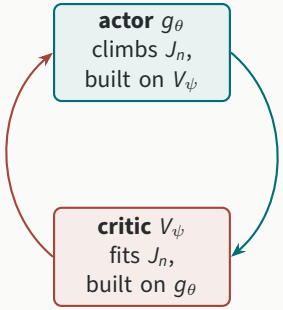


- $n = 1$ is fitted value iteration (p. 20): the actor sees the future only through the critic's slope.
- Larger n: label and objective look n periods ahead, and the critic's weight falls to γ^n; Deck 3.1's T-period backup lengthened only the label.
- No shocks here, so long rollouts add no variance; with shocks, the walk's noise (Decks 3.1, 3.3).
- Sampling: Session 2's band, uniform in $\ln k$. **Ergodic sampling** trains where the economy goes and is blind to rare states.
- **A solution is claimed only where it was trained and tested.**

Source: Session 3 lab, block C: γ of Session 2's rungs 0 and 1.

© Zhigang Feng
Redistribution prohibited without permission

Two Moving Targets, and a Residual That Can Mislead



- Each network's target moves with the other: let the critic move faster, and start both near the answer.
- The pair can agree and be wrong: a critic that values a bad actor correctly makes $V_\psi(k) - u(c) - \gamma V_\psi(k')$ small: consistent, not optimal.
- An Euler residual checks optimality, yet Session 2's lab found one small with a wrong rule, from a careless start.
- Build in shape when theory supplies it: a concave critic leaves the actor one peak in k' to climb.
- **No residual certifies an answer; a benchmark can.**

Source: Session 2 lab, block D.

© Zhigang Feng
Redistribution prohibited without permission

What Each Learns, and How It Improves the Rule

	DQN (p. 8)	**REINFORCE** (p. 10)	**actor–critic** (p. 17)
learns	values $\widehat{Q}_\psi(s,a)$; the rule is greedy in them	a rule $g_\theta(a \mid s)$; a value only as a baseline	a rule g_θ and a value $\widehat{V}_\psi$
improves the rule by	the $\max$ inside its target	score $\times$ (return $-$ baseline)	score $\times$ the surprise δ_t
waits for	one period	the end of the path	one period
data	any rule's, replayed	its own paths	its own transitions
the price	the triad; the $\max$'s optimism	variance	a wrong critic's bias

© Zhaotai Peng • Redistribution prohibited without permission

- The actor–critic itself came three times, the same two learners each time: the model summed over z' (p. 16), no model (p. 17), the model rolled forward n periods (p. 20). What differs is what each step reads.
- **Three ways to carry the future: DQN the value alone, REINFORCE the rule alone, the actor–critic both (Deck 3.1, *Two Questions, Two Learners*).**

Why Economists Should Care: Actor–Critic as Two-Step Estimation

	Two-step estimation	Actor–critic
nuisance parameter	first stage	the critic's value: surprises uncorrelated with its gradient
target parameter	second stage	the actor's rule: zero update at the optimum

- p. 16's actor–critic: **a value fitted to its moment condition, a rule moved along its first-order condition**; interleaved, so the critic must keep ahead: prefitted, then refitted.
- **The saving rule is the estimand; the value function is the nuisance that makes it estimable.** The analogy stops there: none of the asymptotic theory carries over.
- **Grade against an exact solution while you can; report seeds as a range. Next, Deck 3.5: every algorithm on one growth model.**

© Zhusang Feng
Redistribution prohibited without permission

Session 3.5

The Growth Model Across Algorithms



Zhigang Feng

© Zhigang Feng
Redistribution prohibited without permission

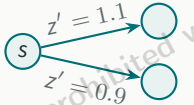
One Model, One Interface

the states, on the grid



one number per (k, z)

one period from s, after choosing k'

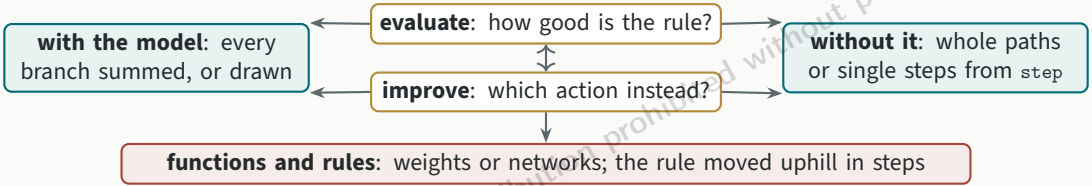


- **State** $s = (k, z)$: capital, and a productivity shock with two values that persists. **Action**: next period's capital k', or the share of output saved. **Reward**: $\ln c$.
- **Three sizes**: a 7×2 grid for the tables; continuous k with the same two shocks for the functions (Deck 3.4); Session 2's deterministic version for deep value iteration.
- **Two calls**: `branches` returns every z' with its probability, `step` one drawn z' and r. From Deck 3.2 on we pretend not to know P_z: learners call only `step`.
- **Every algorithm below learns a value or a rule from one of these two calls.**

Source: Session 3 lab, block A (the two calls); $\alpha = 0.36$, $\gamma = \beta = 0.95$, depreciation 0.10, $u = \ln c$, $z \in \{0.9, 1.1\}$ switching w.p. 0.2.

© Zhiheng Feng · Redistribution prohibited without permission

Every Algorithm Answers Two Questions



© Thiago Pena. Redistribution prohibited without permission

- Every algorithm makes four choices: what it **stores** (a table, a basis, a network); what stands in for the **expectation** over z' (a sum, one draw, a path); how many real periods its **target** counts before it trusts the stored estimate; how it **improves** the rule (a $\max$, ϵ-greedy, a gradient step).
- **Each page: on top what is stored and what one update reads; below, the update, and in bold the one change.**

Session 3.5

With the Model in Hand

Every branch of tomorrow summed with $P_{z'}$ or drawn: Decks 3.1 and 3.4



© Zhigang Feng · Redistribution prohibited without permission

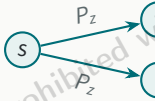
Value Function Iteration and Policy Iteration

stores



one number per (k, z)

one update reads, at every grid point



every z', weighted:
`branches`

© Zhaonan Feng · Redistribution prohibited without permission

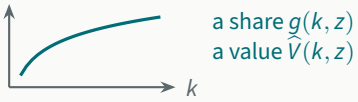
- **Stores** a table $V(k, z)$, one number per grid state. **Reads** u, f and P_z at every grid point: the whole model.
- **One update**, at every grid point, with $c = y(k, z) - k'$:

$$V(k, z) \leftarrow \max_{k'} \left\{ u(c) + \gamma \sum_{z'} P_z(z' \mid z)\, V(k', z') \right\}.$$

- **Improves** by that $\max$: $g(k, z)$ is the best k'; one backup, then improve. Policy iteration instead values the current rule all the way to convergence before improving it: Howard's method.
- **The benchmark: exact, and the answer key for every page after this.**

The Simplest Actor–Critic (Deck 3.4)

stores



one update reads, at states drawn in a band



- **Stores** two functions of continuous k: a saving share $g(k,z)$ and a value $\widehat{V}(k,z)$. **Reads** the model whole, at states drawn afresh in a band.
- **One update** (critic): fit $\widehat{V}$ to the one-period backup

$$B(s) = u(c) + \gamma \sum_{z'} P_z(z' \mid z)\, \widehat{V}^{-}(k', z').$$

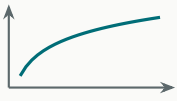
- **Improves** (actor) by one small step: save more where tomorrow's marginal value of capital beats today's marginal utility, less where it falls short. The max is taken in steps, with no search over k'.
- **The first change: no grid, no sweep, no search over k'; the model stays.**

Source: Session 3 lab, block B.

© Zhigang Feng
Redistribution prohibited without permission

Its Variants: One Drawn Branch, and the Walk

stores



the same two functions

one update reads



one drawn z'; u and f still read

Redistribution prohibited without permission

- **Stores** the same two functions. **Reads** u, f and one drawn z', not every z' summed.
- **One update** (critic), toward one drawn branch: right on average, noisy on any one draw:

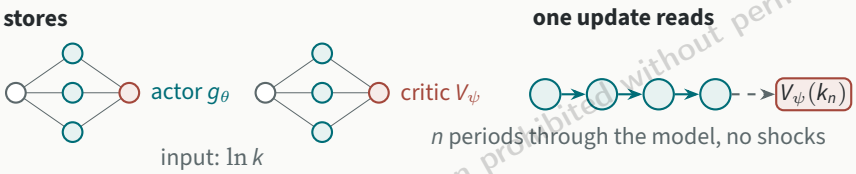
$$\widehat{B}(s) = u(c) + \gamma\, \widehat{V}^{-}(k', z'), \qquad z' \text{ drawn; the actor's step uses the same draw.}$$

- T **periods ahead**, the **walk**: run the rule N times for T periods from s and average, so the critic's target holds T real periods before the estimate.
- **The bridge: a run that ends on $\gamma^T \widehat{V}(s_T)$ is n-step TD; a run to the end is Monte Carlo.**

© Chaoran Feng

Source: Session 3 lab, block B.

Deep Value Iteration (Deck 3.4)



- **Stores** two networks on continuous k: an actor g_θ (the share) and a critic V_ψ. **Reads** Session 2's deterministic model for n periods, at states drawn in a band.
- **Improves** first (actor): one gradient step up on J_n, through all n budget constraints:

$$J_n(k) = \sum_{t<n} \gamma^t u(c_t) + \gamma^n V_\psi(k_n).$$

- **One update** (critic): regress $V_\psi(k)$ on $J_n(k)$, recomputed and held fixed as a label.
- **The second change: both functions networks, both steps n periods deep; no theorem guarantees the pair.**

Source: Session 3 lab, block C.

© Zhigang Feng Registration prohibited without permission

Session 3.5

Without the Model

We pretend not to know P_z: whole paths, single steps, then functions and rules (Decks 3.2–3.4)



© Zhigang Feng · Redistribution Prohibited without permission

Monte Carlo Evaluation





© Zhigang Feng Redistribution prohibited without permission

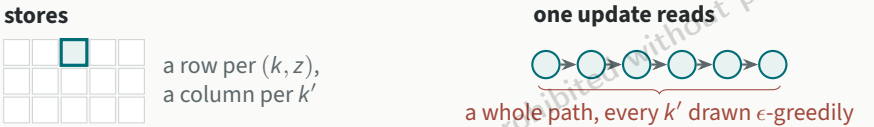
- **Stores** a table $\widehat{Q}(k, z, k')$ of running means, and visit counts C. **Reads** only what `step` returns: rewards and states.
- **One update**, after a whole simulated path under a rule g: each visited cell $x = (k, z, k')$ moves toward the discounted utility G that followed it,

$$\widehat{Q}(x) \leftarrow \widehat{Q}(x) + \big(G - \widehat{Q}(x)\big)/C(x).$$

- **Improves**: not yet; it values a fixed rule g. The luck of one whole path replaces the sum over z'.
- **The third change: simulation replaces integration.**

Source: Session 3 lab, block D.

Monte Carlo Control



- **Stores** $\widehat{Q}(k, z, k')$, visit counts. **Reads** whole paths, every k' drawn by the rule.
- **One update**: the path's returns averaged into $\widehat{Q}$, as on the last page. **Improves** after each path, **ϵ-greedy** in the new $\widehat{Q}$:

$$k' = \arg\max_{a} \widehat{Q}(k, z, a) \text{ with probability } 1 - \epsilon, \text{ a random feasible } k' \text{ otherwise.}$$

- **The fourth change: the rule improves from its estimates and keeps exploring.**

Source: Session 3 lab, block D.

© Zhigang Feng Redistribution prohibited without permission

TD(0)

stores



one number per (k, z)

one update reads



one draw from `step`: r and $s' = (k', z')$

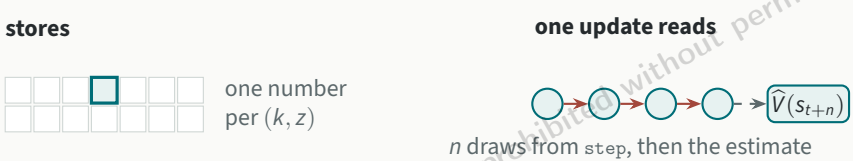
© Zhigang Feng Redistribution prohibited without permission

- **Stores** a table $\widehat{V}(k, z)$ for a fixed rule g, visit counts C. **Reads** one call of `step`.
- **One update**, after each call: today's reward plus the stored value of where it landed, against the stored value of where it was,

$$\widehat{V}(k, z) \leftarrow \widehat{V}(k, z) + \eta\big[r + \gamma \widehat{V}(k', z') - \widehat{V}(k, z)\big].$$

- **Improves**: not yet; it values g. The rest of the path is borrowed from $\widehat{V}$.
- **The fifth change: one real reward, then the estimate. No waiting for the path.**

Between One Step and the Whole Path: n-Step TD and TD(λ)



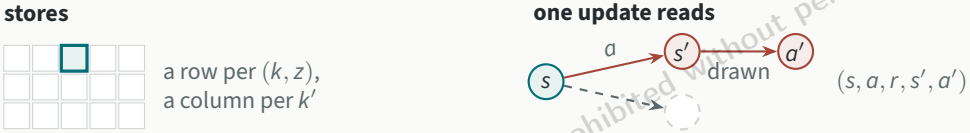
© Zhirang Feng · Redistribution prohibited without permission

- **Stores** $\widehat{V}(k, z)$; TD(λ) adds a memory of recent visits. **Reads** n transitions, or one at a time.
- **One update** (n-step), n periods after t: n real rewards, then the estimate,

$$\widehat{V}(k_t, z_t) \leftarrow \widehat{V}(k_t, z_t) + \eta \Big[\textstyle\sum_{j<n} \gamma^j r_{t+j+1} + \gamma^n \widehat{V}(k_{t+n}, z_{t+n}) - \widehat{V}(k_t, z_t) \Big].$$

- **TD(λ)** averages every n, with weights that fade geometrically, and does it as the path runs: each surprise is credited to the states visited recently, the latest most (Deck 3.3).
- **n and λ: one dial from TD(0) to Monte Carlo; turned up, less bias, more variance.**

SARSA



- **Stores** $\widehat{Q}(k, z, k')$, and acts ϵ-greedily in it. **Reads** one transition and the next action, drawn: (s, a, r, s', a').
- **One update**: draw k'' at (k', z') from the ϵ-greedy rule, take it next, and set

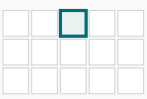
$$\widehat{Q}(k, z, k') \leftarrow \widehat{Q}(k, z, k') + \eta\big[r + \gamma \widehat{Q}(k', z', k'') - \widehat{Q}(k, z, k')\big].$$

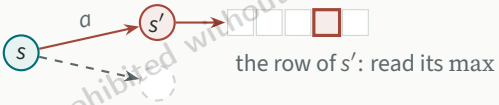
- **Improves** at every step: the rule follows the latest $\widehat{Q}$. It learns the value of the trembling rule it follows; to reach g^*, the trembles must fade slowly (Deck 3.3).
- **The sixth change: control on every step, valuing the rule it follows.**

Source: Session 3 lab, block E.

© Zhuoer Feng Redistribution prohibited without permission

Q-Learning





- **Stores** $\widehat{Q}(k, z, k')$, and acts ϵ-greedily in it. **Reads** (s, a, r, s'); the max is read off the row of (k', z'), nothing simulated.
- **One update**:

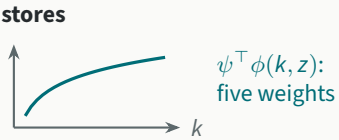
$$\widehat{Q}(k, z, k') \leftarrow \widehat{Q}(k, z, k') + \eta\big[r + \gamma \max_{k''} \widehat{Q}(k', z', k'') - \widehat{Q}(k, z, k')\big].$$

- **Improves** inside the target: the max values tomorrow under the greedy rule, so $\widehat{Q} \to Q^*$ for any fixed $\epsilon > 0$: value function iteration, one sampled transition at a time.
- **The seventh change: one line. The** max **replaces the drawn next action.**

Source: Session 3 lab, block E.

© Zhuoran Feng Redistribution prohibited without permission

Linear TD





© Shuguang Feng · Unauthorized distribution prohibited without permission

- **Stores** five weights ψ: $\widehat{V}_\psi(k,z) = \psi^\top \phi(k,z)$, a weighted sum of five fixed shapes in k and z (Deck 3.4). **Reads** one transition.
- **One update**, moving every state that shares those shapes:

$$\delta = r + \gamma \widehat{V}_\psi(k',z') - \widehat{V}_\psi(k,z), \qquad \psi \leftarrow \psi + \eta\, \delta\, \phi(k,z).$$

- **Improves**: not yet; it values a fixed rule. Where the shapes cannot bend, the fit is wrong at many states at once (Deck 3.4).
- **The eighth change: five weights stand in for the table.**

Source: Session 3 lab, block F.

DQN

stores



$\widehat{Q}_\psi(k, z, \cdot)$: one output per k'; a lagged copy ψ^-; a buffer

one update reads



past (s, a, r, s') in a buffer: a minibatch, drawn

- **Stores** a network $\widehat{Q}_\psi$: input (k, z), one output per k'; a lagged copy ψ^- of its weights; and a buffer of past transitions. **Reads** a minibatch drawn from the buffer.
- **One update**, one gradient step on the minibatch:

$$\min_{\psi} \left(r + \gamma \max_{k''} \widehat{Q}_{\psi^-}(k', z', k'') - \widehat{Q}_\psi(k, z, k')\right)^2, \quad \text{the label from the lagged copy.}$$

- **Improves** as Q-learning does: ϵ-greedy acts and feeds the buffer; ψ is copied into ψ^- every so many steps, so the labels stand still between copies.
- **The ninth change: learned features, held still by a lagged target and replay.**

Source: Session 3 lab, block F.

© Zhigang Feng. Redistribution prohibited without permission

REINFORCE



- **Stores** a rule $g_\theta(k' \mid k, z)$: at each state, a probability for every feasible k', started uniform; and a baseline $\widehat{V}(k, z)$. **Reads** whole paths.
- **One update**, after a whole path under g_θ, with G_t the discounted utility that followed period t:

$$\theta \leftarrow \theta + \eta \sum_t \gamma^t \nabla_\theta \ln g_\theta(k'_t \mid k_t, z_t) \left(G_t - \widehat{V}(k_t, z_t)\right).$$

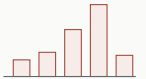
- **Improves** by that step: a k' that did better than the baseline becomes likelier, one that did worse less likely. No search over k', no model.
- **The tenth change: the** $\max$ **is taken in gradient steps on the rule.**

Source: Session 3 lab, block G.

© Zheng Feng · Not distribution prohibited without permission

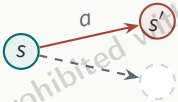
Actor–Critic Without the Model

stores



actor $g_\theta(k' \mid k, z)$
critic $\widehat{V}_\psi(k, z)$

one update reads



one draw from `step`:
r and $s' = (k', z')$

- **Stores** an actor $g_\theta(k' \mid k, z)$, probabilities over k', and a critic $\widehat{V}_\psi(k, z)$. **Reads** one transition; a restart at random now and then spreads the visits.
- **One update** (critic), then the actor, on the same transition and surprise:

$$\delta = r + \gamma \widehat{V}_\psi(k', z') - \widehat{V}_\psi(k, z),$$

$$\psi \leftarrow \psi + \eta_\psi \, \delta \, \nabla_\psi \widehat{V}_\psi(k, z), \qquad \theta \leftarrow \theta + \eta_\theta \, \delta \, \nabla_\theta \ln g_\theta(k' \mid k, z).$$

- **Improves** by the actor's step; each learner has its own step size.
- **The eleventh change: the TD error replaces the whole return.**

Source: Session 3 lab, block G.

© Zhigang Feng
Redistribution prohibited without permission

Session 3.5

Side by Side

Every algorithm against the same questions, and when to use which



© Zhigang Feng Redistribution prohibited without permission

With the Model, Side by Side

algorithm	stores	tomorrow, by	looks ahead	improves by
value function iteration	a table V	every z', summed	one period	a $\max$ over k'
policy iteration	a table V	every z', summed	until the rule's value settles	a $\max$ over k'
the simplest actor–critic (3.4)	two functions	every z', summed	one period	a derivative step
one drawn branch	two functions	one z', drawn	one period	a derivative step
the walk	two functions	the rule run N times	T periods	a derivative step
deep value iteration	two networks	no shocks to draw	n periods, both	a step through n periods

© Anirang Feng · Redistribution prohibited without permission

- All read μ and f. The first three sum over P_z, the next two draw from it, and deep value iteration has no shocks.
- **Down the rows: a draw for the sum, functions for the table, steps for the $\max$.**

Without the Model, Side by Side

algorithm	stores	the target uses	looks ahead	improves by
Monte Carlo	a table $\widehat{Q}$	a whole path's return	to the end	ϵ-greedy, per path
TD(0); n-step, TD(λ)	a table $\widehat{V}$	real rewards, then $\widehat{V}$	1 or n periods	(it evaluates)
SARSA	a table $\widehat{Q}$	a drawn k''	one period	ϵ-greedy, per step
Q-learning	a table $\widehat{Q}$	the row's $\max$	one period	a $\max$ in the target
linear TD	five weights ψ	one reward, then $\psi^\top \phi$	one period	(it evaluates)
DQN	a network, a copy	the $\max$, from the copy	one period	a $\max$ in the target
REINFORCE	a rule θ, a baseline	a whole path's return	to the end	a step on the rule
actor–critic	a rule θ, a critic	one reward, then $\widehat{V}_\psi$	one period	a step on the rule

© Zhaoyang Feng. Unauthorized distribution prohibited without permission

- All read only (s, a, r, s') from `step`. We pretend not to know P_z; in Session 4 the distribution's law of motion is usually truly unknown to the household.
- **Down a column, what changed; along a row, what one algorithm is.**

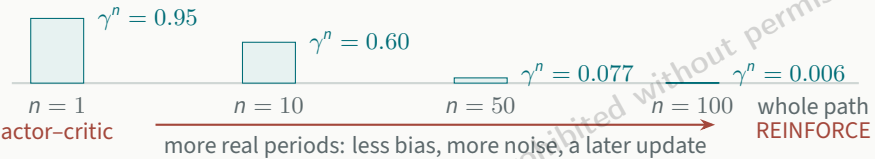
VFI, Q-Learning, Actor–Critic: Which Dynamic Programme Each Samples

	value function iteration	Q-learning	actor–critic
stores	a table $V(k, z)$	a table $\widehat{Q}(k, z, k')$	a rule g_θ and a critic $\widehat{V}_\psi$
values	V^*	Q^*, from any behaviour	V^g of today's rule, which moves
improves by	a $\max$: a jump	a $\max$ inside the target	a step on θ, pushed by δ
data	none: the model	any rule's path (off-policy)	today's rule's path (on-policy)
suits	a small grid, P_z known	a few discrete k'	a share in $(0, 1)$, many actions

- **Q-learning is VFI relaxed three ways**: one drawn z' for the sum, a step $\eta < 1$ for the replacement, the visited cell for the sweep. Restore all three and it is VFI.
- **Actor–critic is policy iteration done partially**: one TD step values today's rule, one gradient step improves it, so its critic chases a moving rule. Q-learning's target ignores behaviour; its data does not.
- **The $\max$ decides: a few k' to compare, Q-learning; a share in $(0, 1)$, actor–critic, climbing in steps.**

© Zhiwei Feng — Unauthorized distribution prohibited without permission

REINFORCE and Actor–Critic: One Dial

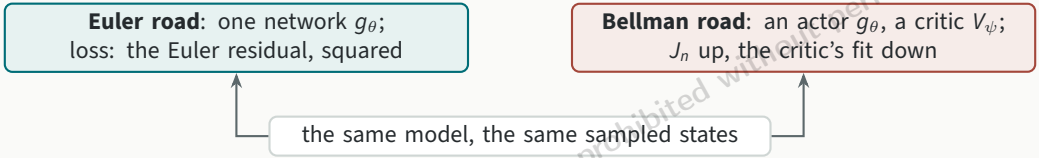


© Zhigang Feng. Redistribution prohibited without permission

- **The question.** Both move the rule by the score. Is actor–critic just a better REINFORCE?
- Credit k'_t with $A_t^{(n)} = \sum_{j<n} \gamma^j r_{t+j+1} + \gamma^n \widehat{V}(s_{t+n}) - \widehat{V}(s_t)$: $n = 1$ is actor–critic; to the path's end it is $G_t - \widehat{V}(s_t)$, REINFORCE with the critic as baseline.
- The critic's error enters scaled by γ^n (the bars); as a baseline alone it adds no bias.
- Each end loses somewhere: at half budget actor–critic collapsed on three seeds of five, REINFORCE on none; deep value iteration with $n = 1$ settled on a wrong rule. Practice averages over n.
- **One question, as for TD: how many real periods before trusting the critic.**

Source: Session 3 lab, blocks C and G (the half-budget pilot).

Euler or Bellman: Two Roads on Session 2's Model



© Miguel Fernandez. Redistribution prohibited without permission

- **Euler** (Session 2): drive $u'(c) - \gamma\, u'(c')\big[f'(k') + 1 - \delta_k\big]$ to zero at sampled states (f' the marginal product, δ_k depreciation). One network, a sharp condition, no value function.
- **Bellman** (Deck 3.4): two networks and no first-order condition, so a kinked value is no obstacle, and a value comes out for welfare comparisons. A discrete choice needs the actor of p. 19, which moves probabilities, not a derivative through the budget.
- **A decision, not a doctrine: take the Euler road when the first-order conditions are clean.**

Source: Session 3 lab, block C.

When Dynamic Programming, When Reinforcement Learning

	expectation computable	expectation only simulable
low dimension	DP: nothing beats it	tabular RL; coverage is the issue
high dimension	approximate DP: deep value iteration	deep RL's home ground

- Bottom left is where most structural economics lives: the model is written, the grid cannot be built. RL's sampled paths and fitted functions can win there too.
- In use: dynamic discrete choice, where Rust re-solves the Bellman equation inside the estimation; high-dimensional macro; interest-rate and tax rules.
- **The caveat: reinforcement learning scales the *solution*, not the *identification*. The data discipline lives in the calibration.**

© Zhiguo Feng. Unauthorized distribution prohibited without permission

RL's One Idea, and the Shared Toolkit

© Yuegang Feng Redistribution prohibited without permission

The substitution	Decks	also in DP, macro	What it costs
simulation for integration	3.2, 3.3	simulated moments	noise; states the paths rarely reach
stored functions for tables	3.4	projection, deep DP	approximation error; with a target built from itself and data from another rule, no guarantee
sampled states where the economy lives, for a sweep	3.1, 3.4	ergodic-set methods	blind to rare states
gradient steps for the search over actions	3.1, 3.4	Euler methods	step sizes and seeds; a pair that agrees can still be wrong
the price: the guarantees	3.1–3.4		an answer key; ranges over seeds

- **RL's own idea: the sampled transition its own rule generates. The rest is a toolkit shared with modern macro computation.** Still owed: an equilibrium, a government, a test.

Session 3.5

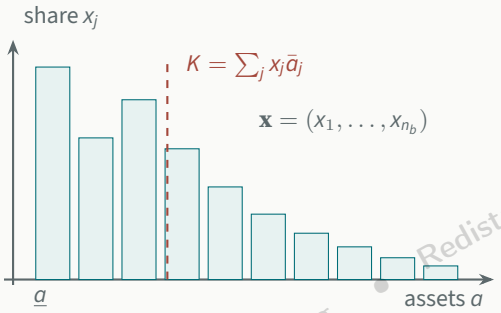
From One Household to an Economy

What Session 4 inherits: the distribution in the state



© Zhigang Feng · Redistribution prohibited without permission

Bridge (1): Put the Distribution in the State, as a Histogram

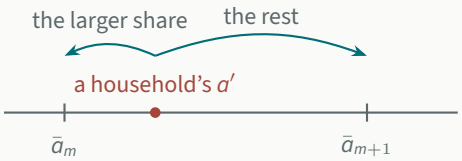


- The method starts by choosing what to carry of the wealth distribution. Krusell and Smith carry its mean. From here a is assets, Session 4's symbol; the choice is a'.
- Carry a **histogram** instead: the share x_j of households in each of n_b asset bins. No shape is assumed; the mass at the borrowing limit and the tail survive.
- Prices follow inside the model: K, then r and w from the firm's conditions.
- **The state is $n_b + 2$ numbers, which a network takes as one input vector.**

© Zhigang Feng
Redistribution prohibited without permission

Source: Method of Feng, Han, Sargent & Zhu (2026, working paper), research in progress.

Bridge (2): an Analytic Transition Matrix, Not a Simulated Panel



its mass is split between the two neighbouring bins, the nearer one getting more

$\mathbf{x}_{t+1} = \mathbf{x}_t \, \mathbf{M}_g$: a matrix built from the rule

- A household's a' lands between two grid points; its mass is split between them by nearness. Summed over households and shocks: tomorrow's histogram, exact, as one matrix $\mathbf{M}_g$.
- It works under *any* rule, as Session 4's government needs, and changes smoothly with the rule; a simulated panel jumps when a household crosses a bin edge.
- **Deck 3.1's promise kept: Γ_g has no closed form, yet a period can always be run, here as one matrix product.**

© Zhigang Feng Redistribution prohibited without permission

Source: The two-point split is Young's (2010); using it as the law of motion a learner trains against: Z. Feng, J. Han, T. Sargent & S. Zhu (2026, working paper), research in progress.

Bridge (3): the Household's Actor–Critic

- **The question.** A household in this economy must forecast prices, so the histogram $\mathbf{x}$ is part of its state. How does it solve its problem with the distribution as an input?
- **The same two learners**, both networks reading $\mathbf{x}$ as one input: a critic $V_\psi(a, z, \mathbf{x})$ and an actor $a' = g_\theta(a, z, \mathbf{x})$, trained at distributions drawn near the current one.
- **The critic's step**: regress V_ψ on the one-period backup at $(a', z', \mathbf{x}')$, the label from a frozen copy of the critic, refreshed at intervals.
- **The actor's step**: one gradient step up the same backup, through the budget constraint (Deck 3.4).
- **The distribution's motion is taken as given**: $\mathbf{x}' = \mathbf{x}\,\mathbf{M}_g$ enters as a number, not differentiated; one household cannot move it. Stop when Euler residuals are small and the pushed-forward aggregates settle.
- **Without results: Session 4 owns equilibrium, the government, validation.**

Source: Feng, Han, Sargent & Zhu (2026, working paper), research in progress.

© Jesse C. Perla · Redistribution prohibited without permission

Using Reinforcement Learning on Your Model: Five Decisions

decide	this session's growth model	Session 4's household
1 the state	(k, z); enlarge it until tomorrow depends on it alone (Deck 3.1)	assets, shock and the distribution: a histogram, moments, or a history
2 the action	grid: k', infeasible ones masked; continuous: a share between 0 and 1	next assets, from an actor network
3 reward, discount	$u(c)$; γ is the economist's β	the same
4 what it may read	the model: sum, draw, roll forward (3.1, 3.4); `step`: sample, explore (3.2–3.4)	the model, written; the law of motion rebuilt each date
5 the grade	an answer key, DP's g^*; seeds as a range	a benchmark where one exists; else residuals, aggregates

© Zhiyang Feng · Redistribution prohibited without permission

- Decisions 1 and 4 are where macro departs: a distribution in the state, and a model written in full whose law of motion for it has no closed form (Deck 3.1).
- **Decide these five first: decision 4 picks the half of this deck you need; decision 5 tells you whether it worked.**

What Session 4 Inherits

Built in Session 3	Used in Session 4
an answer key before any learner	the stationary equilibrium by classical methods first
actor and critic, model known	household policy and value, distribution as an input
gradients through the model	only the government differentiates through $\mathbf{x}'$
a grade in units of consumption	consumption-equivalent welfare across tax regimes
small residual, not a right answer	judge a solution by its simulated aggregates

© Hugget, Fernández-Villaverde, Sargent, Stachurski · Do not distribute or post without permission

- **Every algorithm this session ran on a growth model chosen so that an answer key exists**: a printable grid, a fine dynamic programme, a closed form. The point was to see how each one works and how each is graded; none of it is a finding about an economy.
- **Session 4 starts on the problems these tools are for: Aiyagari, Krusell–Smith, transitions, taxation — where no answer key exists and a solution is judged by its simulated aggregates.**